Poké Fresh

Arquitectura y verificación del backend

Adaptación del Caso 19

Daniel Contreras · Reynner Ontiveros

Frontend de referencia: prototipo responsivo de Poké Fresh

9 de noviembre de 2026

Índice

Alcance y entorno **2**

1 Servicios y contratos **2**

2 Clases y cardinalidades del dominio **17**

3 Sistemas e integraciones **18**

4 Stakeholders y permisos HTTP **20**

5 Clases del API Gateway **22**

6 Rutas y secretos de integración **23**

7 Datos sensibles y secretos del cliente **26**

8 Separación privada y agregaciones GraphQL **27**

9 Analítica y comunicación gRPC cuando correspondan **28**

10 Verificación del backend **29**

Alcance y entorno

Poké Fresh es un sistema de pedidos de bowls personalizados. Adapta las reglas del Caso 19 al catálogo, el pago y el despacho de Poké Fresh.

Versión y entorno. Los resultados de este informe corresponden a la versión local del backend de Poké Fresh, extraída en una carpeta limpia y ejecutada en Windows 11 (build 26200) con Python 3.14.9, Pydantic 2.14.3, MongoDB 8.3.8 y Vault 2.1.1 en modo dev. MongoDB opera como réplica local de un miembro. El Gateway y sus dependencias escuchan en `127.0.0.1`; E01 registra los puertos observados. La verificación se realizó con PowerShell y `curl.exe` contra el Gateway en `http://127.0.0.1:8000`, con acceso directo controlado en T18. Swagger UI y OpenAPI sirven como referencia de los contratos.

Alcance. El sistema se verifica mediante la API. Las pantallas de referencia de Semana 7 se usan para identificar requisitos y operaciones. Según el alcance indicado por el docente, la integración del prototipo con el backend queda fuera de esta evaluación.

Límites del entorno local

Punto	Estado
Vistas de cuenta, checkout y operación	Los flujos se ejecutan mediante la API.
TLS y seguridad de persistencia	HTTP local, Vault dev efímero y MongoDB compartido sin autenticación propia. La autoridad de escritura se aplica en el código de los servicios.
Correo y pago externos	Adaptadores locales. Se acredita canje de códigos, registro del aviso y transiciones de pago, venta y boleta de demostración. La entrega real del correo, la verificación de un buzón externo y el movimiento de dinero quedan pendientes.
Recuperación y caducidad	El reintento retoma pasos persistidos. Queda pendiente reproducir fallos parciales con servicios reales en Windows. Las reservas pendientes carecen de recuperación o caducidad automática.
gRPC y analítica especializada	ADR-01 y ADR-02 justifican HTTP y agregación transaccional. E21 mide la alternativa con dos ventas y concurrencia 1. Queda por medir con el volumen previsto.
Retención y supresión	Baja lógica con conservación de historial. Los códigos temporales caducan. Queda pendiente una política de supresión y retención de los demás datos.
Pruebas automatizadas	Se ejecutó un guion de PowerShell con verificaciones programáticas y revisión de PDF. El código de arranque se entrega junto al backend. Las capturas documentan la ejecución. El guion de esa corrida y una suite independiente quedan fuera de los archivos adjuntos.

1 Servicios y contratos

El Caso 19 define los actores, los datos del registro, la cobertura de 3 km, la confirmación previa a la preparación, la venta, la boleta y el despacho. Las pantallas móviles y de escritorio comparten los contratos descritos aquí. La tabla usa `/v1` como abreviatura de `/api/v1` en las rutas de la matriz; en los ejemplos de pruebas `B` representa `http://127.0.0.1:8000/api/v1`. Las rutas `/auth` y `/health` van directamente bajo el Gateway.

ID	Pantalla o flujo	Acción y requisito
P01	Inicio, Menú y Especiales	R01. Consultar disponibilidad, catálogo, categorías y ofertas. Inicio y navegación son locales.
P02	Arma tu bowl	R02. Consultar opciones, comprobar combinaciones y cotizar.
P03	Mi cuenta y registro	R03. Alta completa, verificar correo, autenticar, recuperar y cerrar sesión. Falta vista de canje y recuperación efectiva.
P04	Confirmar compra y carrito	R04. Cotizar, verificar el radio, crear pedido y comenzar pago. Cantidad, sugerencias y eliminación en el borrador son acciones locales. Falta selector de destino local y confirmación de cajero.
P05	Anular pedido	R05. Consultar pedidos propios y anular con motivo. Vaciar un borrador carece de efecto persistente.
P06	Boleta digital	R06. Consultar boleta de venta y registrar aviso al destinatario. Falta integración de correo real.
P07	Administrador, productos	R07. Consultar, crear y modificar productos, disponibilidad y stock. La tabla original es de lectura.
P08	Administrador, clientes y dashboard	R08. Alta, consulta, modificación y desactivación de clientes. Falta mantenedor de usuarios internos y buzón de laboratorio. El dashboard puede combinar consultas de inventario, clientes, reporte y despacho.
P09	Administrador, reporte	R09. Informe por período y PDF. La pantalla trae un total fijo y una exportación simulada.
P10	Administrador, despacho	R10. Cola de pagados, orden PDF y transiciones. Faltan impresión y controles de estado en la vista.
P11	Brecha transversal	R11. Agregación privada de pedido y pago.
P12	Diagnóstico operativo	R12. Salud, aislamiento y arranque reproducible.

Datos de arranque. Se conservan 9 productos `prod-001` a `prod-009`, las categorías `clasicos`, `proteicos` y `veggie`, tres especiales `esp-001` a `esp-003` con su precio normal y promocional, 4 bases, 4 proteínas, 7 opciones de topping y 4 salsas. El bowl parte en CLP 4.990. `ninguno` conserva su ID y precio cero y excluye otros toppings. Las existencias de laboratorio son 40 unidades por producto/oferta y 60 por ingrediente inventariable. Los productos prediseñados representan existencias de producto terminado y los bowls consumen porciones de ingredientes. El catálogo mantiene sus imágenes como metadatos emoji. Los especiales se publican según `habilitado` y stock, con vigencia administrada de forma explícita.

Datos y reglas revisados. Se descartaron los listados simulados de clientes, despachos y ventas, las credenciales de demostración del navegador, el correlativo local de boleta y la compra de invitado. La regla del caso limita la entrega gratuita al radio completo de 3 km y rechaza el destino fuera de cobertura. Los precios, estados, propietario y stock los determina el backend. Los destinos ficticios están en Puente Alto y el origen de despacho es configurable.

Contratos REST y GraphQL

Los ejemplos de respuesta son fragmentos ilustrativos de contrato. Un GET no lleva cuerpo. El Gateway exige JSON para los cuerpos, con máximo 32 KiB. Campos omitidos en el ejemplo de respuesta pueden existir según Swagger. Para rutas protegidas se usa `Authorization: Bearer TOKEN`. Los campos de entrada son obligatorios salvo los descritos como opcionales o con valor predeterminado. Parámetros de paginación: `after` es el último ID textual y `limit` un entero. IDs de cliente, usuario y pedido se

reciben de respuestas de creación. Errores REST usan `{"detail":"mensaje"}` o, para validación, una lista de ubicación y tipo sin copiar el dato ingresado. Las reglas comunes se detallan en los módulos 4 y 6. La cotización y el pedido aceptan entre 1 y 30 líneas, cada una con cantidad entera de 1 a 30. Una línea de producto contiene su ID; el bowl exige IDs de base, proteína y salsa, con cero a seis toppings distintos. El catálogo admite precios enteros de CLP 0 a 1.000.000. Los servicios rechazan campos JSON fuera de su esquema. Los formatos completos se pueden consultar en OpenAPI desde el Gateway.

ID	Requisito / pantalla	Destino	Operación	Acceso	Datos
S01	R03/P03	auth	POST `/auth/login`	N	S
S02	R03/P03	auth	POST `/auth/logout`	C A K D O	S
S03	R03/P03	auth	POST `/auth/verify`	N	S
S04	R03/P03	auth	POST `/auth/resend`	N	S
S05	R03/P03	auth	POST `/auth/recovery`	N	S
S06	R03/P03	auth	POST `/auth/reset`	N	S
S07	R08/P08	auth	POST `/v1/admin/users`	A	R/S
S08	R08/P08	auth	GET `/v1/admin/users`	A	R
S09	R08/P08	auth	PUT `/v1/admin/users/{user_id}`	A	R
S10	R03/P03	clients	POST `/v1/clients`	N	R/S
S11	R08/P08	clients	POST `/v1/admin/clients`	A	R/S
S12	R03/P03	clients	GET `/v1/clients/me`	C	R
S13	R08/P08	clients	GET `/v1/admin/clients`	A	R
S14	R08/P08	clients	GET `/v1/admin/clients/{client_id}`	A	R
S15	R08/P08	clients	PUT `/v1/admin/clients/{client_id}`	A	R
S16	R08/P08	clients	PUT `/v1/admin/clients/{client_id}/active`	A	R
S17	R03/P03, R06/P06, R08/P08	clients	GET `/v1/admin/mail/{recipient_id}`	A	S
S18	R04/P04	orders	GET `/v1/delivery/zones`	N	P
S19	R04/P04	orders	POST `/v1/orders/quote`	N	P
S20	R04/P04	orders	POST `/v1/orders`	C	R
S21	R05/P05	orders	GET `/v1/orders`	C A	R
S22	R05/P05	orders	GET `/v1/orders/{order_id}`	C A K D O	R
S23	R05/P05	orders	POST `/v1/orders/{order_id}/cancel`	C A	R
S24	R10/P10	orders	GET `/v1/dispatch`	A D	R
S25	R10/P10	orders	PUT `/v1/dispatch/{order_id}/state`	A D	R
S26	R10/P10	orders	GET `/v1/dispatch/{order_id}/print`	A D	R
S27	R04/P04	payments	POST `/v1/orders/{order_id}/payment`	C	R

ID	Requisito / pantalla	Destino	Operación	Acceso	Datos
S28	R04/P04	payments	GET `/v1/orders/{order_id}/payment`	C A K O	R
S29	R06/P06	payments	POST `/v1/orders/{order_id}/payment/confirm`	K	R
S30	R06/P06	payments	GET `/v1/orders/{order_id}/receipt`	C A K	R
S31	R09/P09	payments	GET `/v1/reports/sales`	A O	R
S32	R09/P09	payments	GET `/v1/reports/sales/print`	A O	R
S33	R11/P11	graphql	POST `/v1/graphql`	C A K D O	R
S34	R01/P01	inventory	GET `/v1/catalog`	N	P
S35	R02/P02	inventory	GET `/v1/bowl-options`	N	P
S36	R07/P07	inventory	GET `/v1/admin/inventory`	A O	R
S37	R07/P07	inventory	POST `/v1/admin/products`	A O	R
S38	R07/P07	inventory	PUT `/v1/admin/products/{item_id}`	A O	R
S39	R07/P07	inventory	PUT `/v1/admin/inventory/{item_id}/availability`	A O	R
S40	R07/P07	inventory	POST `/v1/admin/inventory/{item_id}/adjustments`	A O	R
S41	R03/P03	gateway	GET `/auth/me`	C A K D O	R
S42	R12/P12	gateway	GET `/health`	N	P

Ejemplos de contrato por operación

S01 POST `/auth/login`

Correo y clave; sesión de 900 s. 401 credenciales, 403 cuenta sin verificar. *Entrada JSON*

```
{
  "email": "cliente@example.test",
  "password": "Demo-segura-2026!"
}
```

Ejemplo JSON de salida

```
{
  "access_token": "TOKEN_OPACO",
  "token_type": "bearer",
  "expires_in": 900
}
```

S02 POST `/auth/logout`

Revoca la sesión Bearer actual; 204 vacío.

S03 POST `/auth/verify`

Código de un uso de 43 caracteres; 400 vencido o consumido. *Entrada JSON*

```
{
  "token": "CODIGO_RECIBIDO_EN_EL_BUZON"
}
```

Ejemplo JSON de salida

```
{
  "verified": true,
  "adapter": "local"
}
```

S04 POST `/auth/resend`

Autentica por correo y clave, renueva el código si queda pendiente; 202 o 401. *Entrada JSON*

```
{
  "email": "cliente@example.test",
  "password": "Demo-segura-2026!"
}
```

Ejemplo JSON de salida

```
{
  "accepted": true
}
```

S05 POST `/auth/recovery`

Correo normalizado; 202 uniforme. El adaptador genera un mensaje sólo si la cuenta es elegible. *Entrada JSON*

```
{
  "email": "cliente@example.test"
}
```

Ejemplo JSON de salida

```
{
  "accepted": true
}
```

S06 POST `/auth/reset`

Código y nueva clave. 204 e invalida sesiones; 400 código vencido o usado. *Entrada JSON*

```
{
  "token": "CODIGO_RECIBIDO_EN_EL_BUZON",
  "password": "Nueva-clave-demo!"
}
```

S07 POST `/v1/admin/users`

Alta interna con rol permitido, clave y correo único por cuenta. 201 o 409. *Entrada JSON*

```
{
  "email": "cajero@example.test",
  "password": "Demo-segura-2026!",
  "nombre": "Cajero Ejemplo",
  "role": "cajero"
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "role": "cajero",
  "active": true,
  "verified": true
}
```

S08 GET `/v1/admin/users`

Lista interna. after opcional; limit 1–100, predeterminado 50. *Ejemplo JSON de salida*

```
[
  {
    "id": "0123456789abcdef0123456789abcdef",
    "nombre": "Cajero Ejemplo",
    "role": "cajero",
    "active": true,
    "verified": true
  }
]
```

S09 PUT `/v1/admin/users/{user_id}`

Reemplaza nombre, rol y activo. Revoca sesiones; 409 si el admin se quita su propio acceso. *Entrada JSON*

```
{
  "nombre": "Cajero Ejemplo",
  "role": "cajero",
  "active": false
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "role": "cajero",
  "active": false
}
```

S10 POST `/v1/clients`

Alta con RUN válido, datos completos y clave. 201 pendiente, 409 correo o RUN ocupado. *Entrada JSON*

```
{
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100",
  "comuna": "Puente Alto",
  "provincia": "Cordillera",
  "region": "Metropolitana",
  "nacimiento": "1997-01-15",
  "sexo": "otro",
  "telefono": "+56 9 1234 5678",
  "email": "cliente@example.test",
  "password": "Demo-segura-2026!",
  "run": "12.345.678-5"
}
```

Ejemplo JSON de salida


```
{
  "id": "0123456789abcdef0123456789abcdef",
  "verification_pending": true,
  "adapter": "local"
}
```


S11 POST `/v1/admin/clients`

Mismas validaciones y verificación que el alta directa; 201 pendiente. *Entrada JSON*


```
{
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100",
  "comuna": "Puente Alto",
  "provincia": "Cordillera",
  "region": "Metropolitana",
  "nacimiento": "1997-01-15",
  "sexo": "otro",
  "telefono": "+56 9 1234 5678",
  "email": "cliente@example.test",
  "password": "Demo-segura-2026!",
  "run": "12.345.678-5"
}
```


Ejemplo JSON de salida


```
{
  "id": "0123456789abcdef0123456789abcdef",
  "verification_pending": true,
  "adapter": "local"
}
```


S12 GET `/v1/clients/me`

Lee perfil del propio titular, incluido RUN y estado de verificación. *Ejemplo JSON de salida*


```
{
  "id": "0123456789abcdef0123456789abcdef",
  "email": "cliente@example.test",
  "run": "12345678-5",
  "verified": true,
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100",
  "comuna": "Puente Alto",
  "provincia": "Cordillera",
  "region": "Metropolitana",
  "nacimiento": "1997-01-15",
  "sexo": "otro",
  "telefono": "+56912345678"
}
```


S13 GET `/v1/admin/clients`

Lista resumida con conteo de pedidos. after opcional y limit 1–100, por defecto 50. *Ejemplo JSON de salida*


```
[
  {
    "id": "0123456789abcdef0123456789abcdef",
```

```
    "nombre": "Cliente Ejemplo",
    "email": "cliente@example.test",
    "pedidos": 0,
    "verified": true,
    "active": true
  }
]
```

S14 GET `/v1/admin/clients/{client_id}`

Detalle de cliente. 404 ID ausente o cuenta interna. *Ejemplo JSON de salida*

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "email": "cliente@example.test",
  "run": "12345678-5",
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100",
  "comuna": "Puente Alto",
  "provincia": "Cordillera",
  "region": "Metropolitana",
  "nacimiento": "1997-01-15",
  "sexo": "otro",
  "telefono": "+56912345678"
}
```

S15 PUT `/v1/admin/clients/{client_id}`

Reemplaza datos de contacto y demográficos. RUN y correo son identificadores estables. *Entrada JSON*

```
{
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100",
  "comuna": "Puente Alto",
  "provincia": "Cordillera",
  "region": "Metropolitana",
  "nacimiento": "1997-01-15",
  "sexo": "otro",
  "telefono": "+56 9 1234 5678"
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "nombre": "Cliente Ejemplo",
  "direccion": "Pasaje Demo 100"
}
```

S16 PUT `/v1/admin/clients/{client_id}/active`

Activa o desactiva; revoca sesiones si cambia. Conserva historial. *Entrada JSON*

```
{
  "active": false
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "active": false
}
```

S17 GET `/v1/admin/mail/{recipient_id}`

Buzón local por destinatario; hasta 50 mensajes, códigos vigentes y avisos de boleta. *Ejemplo JSON de salida*

```
[
  {
    "kind": "verify",
    "adapter": "local",
    "code": "CODIGO_RECIBIDO_EN_EL_BUZON"
  }
]
```

S18 GET `/v1/delivery/zones`

Devuelve origen, radio y direcciones ficticias reconocidas por el adaptador. *Ejemplo JSON de salida*

```
{
  "radius_km": 3,
  "adapter": "local",
  "destinos": [
    {
      "destino_id": "cobertura-1",
      "in_coverage": true,
      "shipping": 0
    }
  ]
}
```

S19 POST `/v1/orders/quote`

Calcula con inventario; 422 ID, combinación o cobertura; 409 stock o ítem inactivo. *Entrada JSON*

```
{
  "items": [
    {
      "tipo": "producto",
      "id": "prod-001",
      "cantidad": 1
    },
    {
      "tipo": "bowl",
      "base": "arroz",
      "proteina": "tofu",
      "toppings": [
        "ninguno"
      ],
      "salsa": "soya",
      "cantidad": 1
    }
  ],
  "destino_id": "cobertura-1"
}
```

Ejemplo JSON de salida

```
{
  "total": 13180,
  "shipping": 0,
  "currency": "CLP"
}
```

S20 POST `/v1/orders`

Cliente activo y verificado; 201 reserva; cabecera Idempotency-Key (8–64 caracteres A–Z, a–z, 0–9, guion o subrayado). 409 por clave usada con otro cuerpo. *Entrada JSON*

```
{
  "items": [
    {
      "tipo": "producto",
      "id": "prod-001",
      "cantidad": 1
    },
    {
      "tipo": "bowl",
      "base": "arroz",
      "proteina": "tofu",
      "toppings": [
        "ninguno"
      ],
      "salsa": "soya",
      "cantidad": 1
    }
  ],
  "destino_id": "cobertura-1"
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "state": "pendiente_pago",
  "total": 13180,
  "currency": "CLP",
  "shipping": 0
}
```

S21 GET `/v1/orders`

Historial propio o global para admin. Omite registros provisionales. after opcional; limit 1–100, predeterminado 30. *Ejemplo JSON de salida*

```
[
  {
    "id": "0123456789abcdef0123456789abcdef",
    "state": "pendiente_pago",
    "total": 13180,
    "currency": "CLP",
    "shipping": 0
  }
]
```

S22 GET `/v1/orders/{order_id}`

Propio para cliente; despacho desde pagado y en estados operativos. Dueño/cajero reciben una proyección. 404 por falta de acceso al recurso. *Ejemplo JSON de salida*

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "state": "pendiente_pago",
  "total": 13180,
  "currency": "CLP",
  "shipping": 0
}
```

S23 POST `/v1/orders/{order_id}/cancel`

Motivo de 3–300 caracteres. Propietario o admin antes de preparar. Repetir conserva el primer motivo. 409 en otro estado o durante una confirmación en curso. *Entrada JSON*

```
{
  "motivo": "Cambio de planes"
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "state": "anulado",
  "cancellation": {
    "motivo": "Cambio de planes"
  }
}
```

S24 GET `/v1/dispatch`

Cola ordenada por hora de confirmación, con desempate por ID. limit 1–100, por defecto 50. *Ejemplo JSON de salida*

```
[
  {
    "id": "0123456789abcdef0123456789abcdef",
    "state": "pagado",
    "cliente": "Cliente Ejemplo"
  }
]
```

S25 PUT `/v1/dispatch/{order_id}/state`

Estado preparando, en_camino o entregado, con predecesor obligatorio. 409 sin pago o transición inválida. *Entrada JSON*

```
{
  "estado": "preparando"
}
```

Ejemplo JSON de salida

```
{
  "id": "0123456789abcdef0123456789abcdef",
  "state": "preparando"
}
```

S26 GET `/v1/dispatch/{order_id}/print`

PDF imprimible, HTTP 200 application/pdf. 409 si aún no está pagado o se anuló.

S27 POST `/v1/orders/{order_id}/payment`

Pedido propio pendiente. webpay o transferencia; escenario_local opcional aprobado/rechazado. 409 ante autorización incompatible. *Entrada JSON*

```
{
  "metodo": "webpay",
  "escenario_local": "aprobado"
}
```

Ejemplo JSON de salida


```
{
  "id": "0123456789abcdef0123456789abcdef",
  "status": "authorized",
  "metodo": "webpay",
  "amount": 13180,
  "adapter": "local"
}
```


S28 GET `/v1/orders/{order_id}/payment`

Propio para cliente. Referencia local sólo para cajero. Sin pago devuelve not_started. *Ejemplo JSON de salida*


```
{
  "id": "0123456789abcdef0123456789abcdef",
  "status": "authorized",
  "metodo": "webpay",
  "amount": 13180,
  "adapter": "local",
  "referencia": "local-0123456789abcdef0123456789abcdef"
}
```


S29 POST `/v1/orders/{order_id}/payment/confirm`

Confirma referencia autorizada y genera venta/boleta. Repetición 200 con duplicate=true. 409 si el adaptador no autoriza. *Entrada JSON*


```
{
  "referencia": "local-0123456789abcdef0123456789abcdef"
}
```


Ejemplo JSON de salida


```
{
  "payment_status": "confirmed",
  "sale_id": "0123456789abcdef0123456789abcdef",
  "receipt_id": "PF-0123456789abcdef0123456789abcdef",
  "duplicate": false
}
```


S30 GET `/v1/orders/{order_id}/receipt`

Boleta propia para cliente; 404 hasta emisión. Devuelve anulación cuando corresponde. *Ejemplo JSON de salida*


```
{
  "number": "PF-0123456789abcdef0123456789abcdef",
  "order_id": "0123456789abcdef0123456789abcdef",
  "total": 13180,
  "document_type": "boleta_demo"
}
```


S31 GET `/v1/reports/sales`

desde y hasta son fechas ISO UTC; intervalo [desde, hasta), hasta 366 días. 422 rango inválido. *Ejemplo JSON de salida*

```
{
  "currency": "CLP",
  "gross": 13180,
  "voided": 0,
  "net": 13180,
  "orders": 1,
  "average_ticket": 13180
}
```


S32 GET `/v1/reports/sales/print`

Mismos parámetros del reporte; 200 application/pdf.

S33 POST `/v1/graphql`

Una consulta, con variables de texto opcionales. 400 documento no permitido; 422 si supera 2.500 caracteres; ejecución con errores de campo/propiedad retorna 200 con errors. *Entrada JSON*


```
{
  "query": "query($id:ID!){pedido(id:$id){id estado total pago{estado monto}}}",
  "variables": {
    "id": "0123456789abcdef0123456789abcdef"
  }
}
```


Ejemplo JSON de salida


```
{
  "data": {
    "pedido": {
      "id": "0123456789abcdef0123456789abcdef",
      "estado": "pagado",
      "total": 13180,
      "pago": {
        "estado": "confirmed",
        "monto": 13180
      }
    }
  }
}
```


S34 GET `/v1/catalog`

categoria opcional; available_only booleano, true por defecto. Incluye ofertas en especiales. *Ejemplo JSON de salida*


```
{
  "moneda": "CLP",
  "products": [
    {
      "id": "prod-001",
      "categoria": "clasicos",
      "precio": 6990,
      "disponible": true
    }
  ]
}
```


S35 GET `/v1/bowl-options`

Precio base y opciones por grupo; cada opción incluye disponibilidad. *Ejemplo JSON de salida*

```
{
  "precioBase": 4990,
  "moneda": "CLP",
  "options": {
    "base": [
      {
        "id": "arroz",
        "precio": 0,
        "disponible": true
      }
    ]
  }
}
```

S36 GET `/v1/admin/inventory`

Stock y habilitación interna; after opcional; limit 1–200, por defecto 100. *Ejemplo JSON de salida*

```
[
  {
    "id": "prod-001",
    "stock": 40,
    "habilitado": true,
    "disponible": true
  }
]
```

S37 POST `/v1/admin/products`

Crea producto con ID único, stock inicial entero y precio CLP. 201 o 409 por ID existente. *Entrada JSON*

```
{
  "id": "prod-demo",
  "stock": 5,
  "nombre": "Bowl Demostración",
  "categoria": "veggie",
  "descripcion": "Tofu, arroz y mango",
  "precio": 6500,
  "habilitado": true
}
```

Ejemplo JSON de salida

```
{
  "id": "prod-demo",
  "stock": 5,
  "disponible": true
}
```

S38 PUT `/v1/admin/products/{item_id}`

Reemplaza datos del producto; conserva stock. Oferta requiere precioNormal mayor que precio. *Entrada JSON*

```
{
  "nombre": "Bowl Demostración",
  "categoria": "veggie",
  "descripcion": "Tofu, arroz y mango",
  "precio": 6500,
  "habilitado": true
}
```

Ejemplo JSON de salida

```
{
  "id": "prod-demo",
  "precio": 6500,
  "disponible": true
}
```

S39 PUT `/v1/admin/inventory/{item_id}/availability`

Cambia habilitado. La disponibilidad también depende de stock positivo. *Entrada JSON*

```
{
  "habilitado": false
}
```

Ejemplo JSON de salida

```
{
  "id": "prod-demo",
  "habilitado": false,
  "disponible": false
}
```

S40 POST `/v1/admin/inventory/{item_id}/adjustments`

Ajuste de stock entero, -100000 a 100000. 409 si el resultado es inválido o el ítem no tiene stock. *Entrada JSON*

```
{
  "delta": 2
}
```

Ejemplo JSON de salida

```
{
  "id": "prod-demo",
  "stock": 7
}
```

S41 GET `/auth/me`

Introspección de la sesión actual, devuelve ID y rol vigente. *Ejemplo JSON de salida*

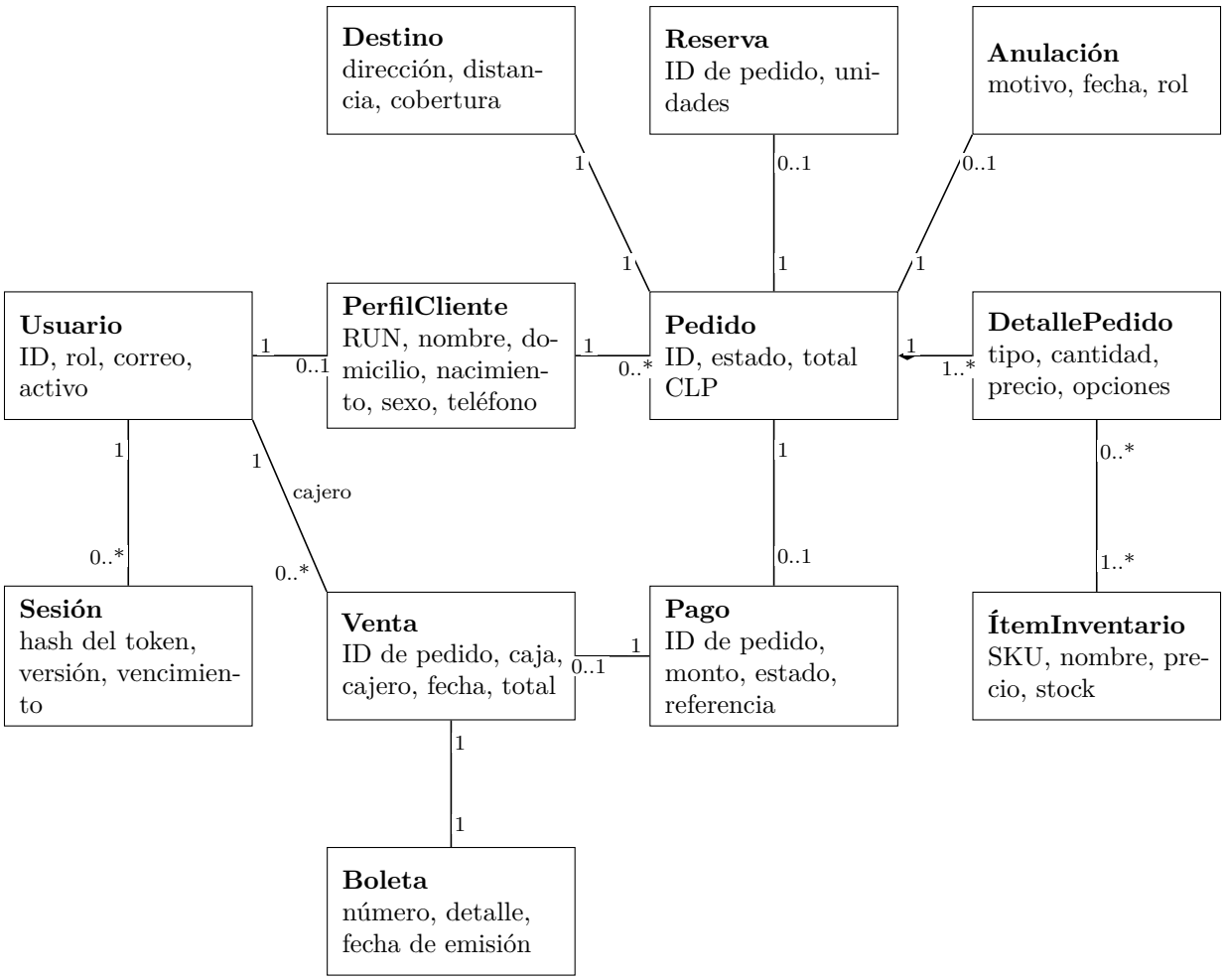
```
{
  "id": "0123456789abcdef0123456789abcdef",
  "role": "cliente"
}
```

S42 GET `/health`

200 si todos los servicios responden, 503 si alguno está degradado. *Ejemplo JSON de salida*

```
{
  "status": "ok",
  "services": {
    "auth": "ok",
    "inventory": "ok",
    "orders": "ok",
    "clients": "ok",
    "payments": "ok",
    "graphql": "ok"
  }
}
```

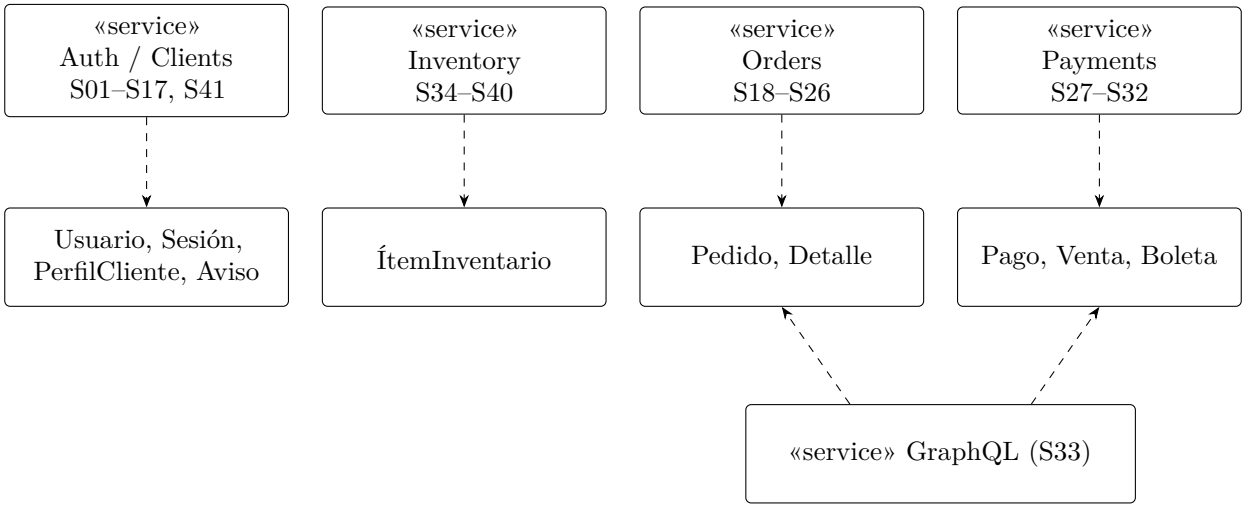
2 Clases y cardinalidades del dominio



Asociación	Multiplicidad	Regla
Usuario – PerfilCliente	1 a 0..1	La cuenta cliente conserva el perfil inicial. Clients guarda las actualizaciones por ID de cliente y, si no existen, usa el perfil inicial. Los usuarios internos carecen de perfil de cliente.
Usuario – Sesión	1 a 0..*	Una cuenta puede iniciar varias sesiones. Cada sesión tiene un titular y una caducidad.
PerfilCliente – Pedido	1 a 0..*	Un titular puede comprar muchas veces. Cada pedido tiene un cliente verificado al crear.
Pedido – DetallePedido	1 a 1..*	Composición en el pedido visible. Tras reservar, contiene al menos una línea y conserva precios y nombres. El registro provisional no se expone.
DetallePedido – ÍtemInventario	0..* a 1..*	Una línea de catálogo refiere a un SKU. Un bowl refiere a base, proteína, salsa y toppings opcionales. El histórico usa IDs y copias de los datos.
Pedido – Reserva	1 a 0..1	Inventory registra una reserva por ID de pedido en el flujo nuevo. Los pedidos anteriores pueden conservar las unidades en su propio documento para permitir la reposición.

Asociación	Multiplicidad	Regla
Pedido – Pago	1 a 0..1	Un documento de pago por pedido. Un intento rechazado puede sustituirse. Una anulación sin intento puede dejar un marcador privado que bloquea intentos tardíos.
Pago – Venta – Boleta	1 a 0..1, 1 a 1	El pago confirmado origina una venta, asociada a un cajero interno y a CAJA-WEB. Toda venta tiene boleta. Los registros se conservan al anular.
Pedido – Destino y anulación	1 a 1, 1 a 0..1	El destino se embebe al crear. La primera anulación guarda su motivo y fecha; la repetición devuelve ese mismo resultado.
Venta – Usuario cajero	0..* a 1	El ID del cajero autenticado se conserva en la venta aunque la cuenta cambie de estado.

Notación y DTO. El rombo representa composición. Las líneas continuas representan asociaciones y la flecha discontinua indica uso. El carrito es un borrador del navegador y su cuerpo de checkout es un DTO. Pedido, pago, cliente y producto tienen DTO que omiten campos internos según rol.



S41 es la consulta de identidad y S42 es la salud del Gateway. Los nombres de operaciones y sus rutas figuran en el catálogo del módulo 1. Las dependencias entre interfaces y entidades no establecen cardinalidad de registros. Inventory es el único escritor de la reserva y el stock. La tabla de asociaciones incluye la cardinalidad de la reserva.

3 Sistemas e integraciones

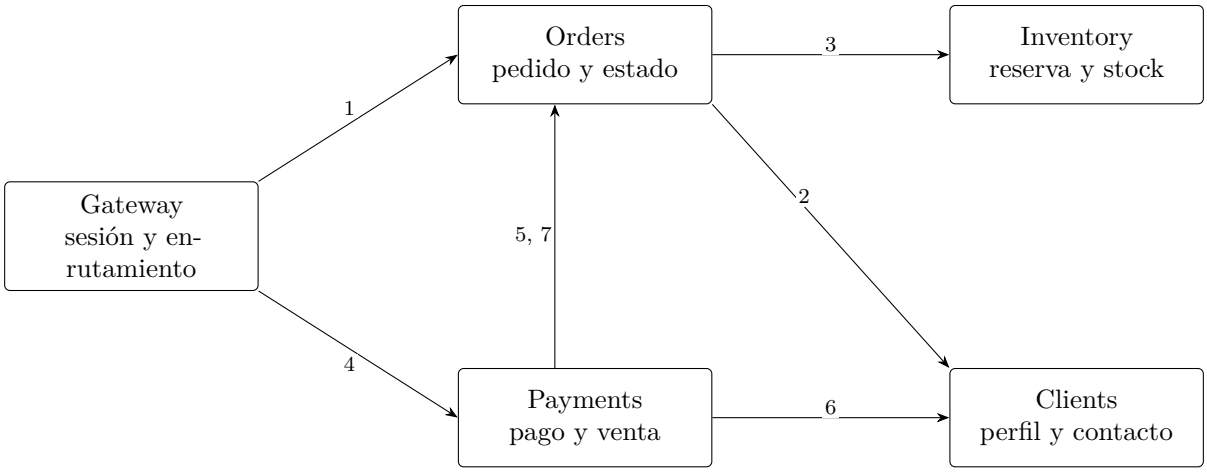
Sistema	Datos que modifica	Colaboración y servicios
Inventory	Catálogo, configuración del bowl, stock y reservas por ID de pedido.	S34–S40. Responde a Orders con cotización, reserva o reposición. Stock insuficiente devuelve 409.
Orders	Pedido, destino, detalle, estado y marcadores de operación.	S18–S26. Valida cobertura, consulta a Clients y coordina Inventory y Payments. Sirve el pedido, su conteo y los cambios de estado por rutas privadas.

Sistema	Datos que modifica	Colaboración y servicios
Clients	Actualizaciones de perfiles.	S10–S17. Delega alta y activación a Auth. Consulta el conteo a Orders y proyecta datos para checkout y aviso de venta. Compone el buzón local a partir de avisos de cuenta y venta.
Payments	Intento, pago, venta, boleta y aviso de venta.	S27–S32. Lee el pedido y solicita a Orders los cambios de estado. Consulta a Clients el contacto del recibo. La venta, boleta y aviso se guardan juntos en su transacción local.
Auth	Identidad, sesiones y códigos de cuenta. Conserva el perfil inicial como respaldo de lectura.	S01–S09; introspección para S41. Atiende alta y activación solicitadas por Clients. El cambio de activación invalida las sesiones anteriores.
GraphQL	Agregación de lectura, sin persistencia propia.	S33. Consulta S22 y S28 por HTTP privado con identidad verificada.
Vault	Credenciales técnicas y políticas KV v2.	El iniciador crea un token acotado a la ruta KV de cada proceso. Sin la carga inicial de secretos, la aplicación no arranca.

Los dominios se ejecutan como procesos HTTP separados y comparten una instancia MongoDB. La autoridad de escritura está separada en el código de aplicación. Clients conserva lecturas del documento de identidad y de la bandeja histórica y de venta. La instancia de laboratorio no impone aislamiento por usuario de base de datos. La réplica de un miembro permite transacciones locales, sin conmutación automática. Las operaciones entre servicios se coordinan mediante pasos persistidos y reintentos, sin una transacción MongoDB única que abarque la compra o la anulación completas.

Secuencia de compra y confirmación

El Gateway verifica la sesión en Auth y envía cada operación a su destino. Las flechas muestran llamadas privadas posteriores al enrutamiento. La tabla indica el orden de los pasos.



Paso	Operación
1	Cliente solicita checkout por el Gateway. Orders valida la cobertura y asocia cliente, clave de idempotencia y huella del cuerpo.

Paso	Operación
2	Orders consulta a Clients los datos de contacto y el estado del cliente. Orders crea un registro provisional para el checkout.
3	Inventory calcula precios, registra la reserva y descuenta stock en su propia transacción. Orders incorpora el detalle y publica el pedido como `pendiente_pago`.
4	Cliente inicia un intento de pago local. El cajero consulta la referencia e inicia la confirmación por el Gateway.
5	Payments consulta a Orders por el pedido y comprueba referencia, monto y estado del intento.
6	Si corresponde, Payments consulta a Clients por el destinatario del aviso y solicita a Orders registrar la confirmación en curso. Payments registra el pago, la venta, la boleta y el aviso en su transacción local.
7	Payments solicita a Orders pasar el pedido a `pagado`. Despacho puede consultar la cola y mover el pedido a `preparando`, `en_camino` y `entregado`.

Anulación. Orders conserva el primer motivo y bloquea el despacho mientras la operación está en curso. Solicita a Payments cerrar el intento y, si hubo venta, marcar el reembolso local y anular venta y boleta. Luego Inventory libera la reserva una sola vez. Orders marca el pedido como `anulado` al completar esos pasos.

Reintentos y fallos parciales. Ante una interrupción, se repite el checkout con la misma clave y cuerpo, la confirmación con la misma referencia o la anulación del mismo pedido. Los pasos ya aplicados pueden reutilizarse. El registro `reserva_en_curso` se omite del historial hasta publicar el pedido. Si Inventory alcanzó a descontar y nadie reintenta, la reserva puede seguir pendiente. No hay recuperación ni caducidad automática. E06, E09, E12, E20 y E22 acreditan reintentos, persistencia y concurrencia en Windows. Los fallos entre pasos se verificaron en pruebas aisladas simuladas y queda pendiente su reproducción con los servicios reales. En las pruebas aisladas se simuló la pérdida de la respuesta tras reservar, una interrupción después de persistir la venta y antes de marcar el pedido como pagado, y la pérdida de respuesta tras reponer stock. Al repetir se completó cada operación sin duplicar el descuento, la venta ni la reposición.

4 Stakeholders y permisos HTTP

Actor	Rol	Interés y alcance
Visitante	N	Catálogo, opciones, cotización, registro y flujos de cuenta.
Cliente registrado	C	Compra, perfil propio, historial, pago propio, anulación y boleta propia.
Administrador	A	Mantiene clientes, usuarios, productos y stock. Ve ventas, despacho y buzón local, y puede anular compras elegibles.
Cajero virtual	K	Confirma el pago y emite la venta asociada a CAJA-WEB. Ve referencia, estado, total y boleta.
Encargado de despacho/bodega	D	Consulta cola, descarga orden y mueve el pedido después del pago.
Dueño	O	Consulta ventas por período y mantiene productos y disponibilidad.
Cocina y choferes	—	Sin permisos operativos. Reciben orden imprimible del encargado.

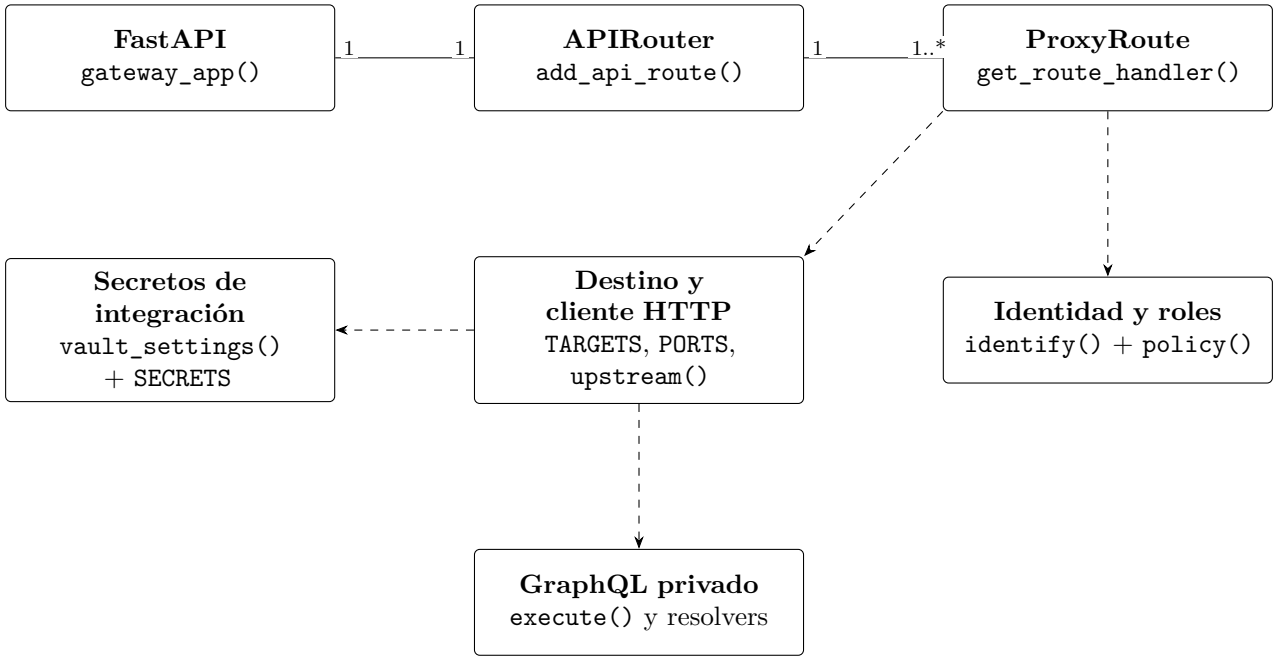
Actor	Rol	Interés y alcance
Proveedor de pago y correo	—	Identidades externas sin permiso operativo en esta versión. Se representan por adaptadores locales.
Gateway, GraphQL y servicios	Técnica	Credenciales de integración acotadas a los destinos descritos. No equivalen a roles humanos.

Lectura de la matriz. Las rutas sin marca para un rol exigen otra identidad. Las rutas abiertas (N) permiten la operación anónima, incluso a un usuario ya conectado. El ID S01–S42 y el verbo identifican un contrato concreto del módulo 1. Para todos los demás verbos sobre la misma ruta se aplica 405 o 404 según el enrutamiento. DELETE carece de operaciones en esta versión. La baja de cuentas y la disponibilidad son PUT y mantienen historial.

ID	HTTP	Ruta	N	C	A	K	D	O
S01	POST	`/auth/login`	✓	—	—	—	—	—
S02	POST	`/auth/logout`	—	✓	✓	✓	✓	✓
S03	POST	`/auth/verify`	✓	—	—	—	—	—
S04	POST	`/auth/resend`	✓	—	—	—	—	—
S05	POST	`/auth/recovery`	✓	—	—	—	—	—
S06	POST	`/auth/reset`	✓	—	—	—	—	—
S07	POST	`/v1/admin/users`	—	—	✓	—	—	—
S08	GET	`/v1/admin/users`	—	—	✓	—	—	—
S09	PUT	`/v1/admin/users/{user_id}`	—	—	✓	—	—	—
S10	POST	`/v1/clients`	✓	—	—	—	—	—
S11	POST	`/v1/admin/clients`	—	—	✓	—	—	—
S12	GET	`/v1/clients/me`	—	✓	—	—	—	—
S13	GET	`/v1/admin/clients`	—	—	✓	—	—	—
S14	GET	`/v1/admin/clients/{client_id}`	—	—	✓	—	—	—
S15	PUT	`/v1/admin/clients/{client_id}`	—	—	✓	—	—	—
S16	PUT	`/v1/admin/clients/{client_id}/active`	—	—	✓	—	—	—
S17	GET	`/v1/admin/mail/{recipient_id}`	—	—	✓	—	—	—
S18	GET	`/v1/delivery/zones`	✓	—	—	—	—	—
S19	POST	`/v1/orders/quote`	✓	—	—	—	—	—
S20	POST	`/v1/orders`	—	✓	—	—	—	—
S21	GET	`/v1/orders`	—	✓	✓	—	—	—
S22	GET	`/v1/orders/{order_id}`	—	✓	✓	✓	✓	✓
S23	POST	`/v1/orders/{order_id}/cancel`	—	✓	✓	—	—	—
S24	GET	`/v1/dispatch`	—	—	✓	—	✓	—
S25	PUT	`/v1/dispatch/{order_id}/state`	—	—	✓	—	✓	—
S26	GET	`/v1/dispatch/{order_id}/print`	—	—	✓	—	✓	—
S27	POST	`/v1/orders/{order_id}/payment`	—	✓	—	—	—	—
S28	GET	`/v1/orders/{order_id}/payment`	—	✓	✓	✓	—	✓
S29	POST	`/v1/orders/{order_id}/payment/confirm`	—	—	—	✓	—	—
S30	GET	`/v1/orders/{order_id}/receipt`	—	✓	✓	✓	—	—
S31	GET	`/v1/reports/sales`	—	—	✓	—	—	✓
S32	GET	`/v1/reports/sales/print`	—	—	✓	—	—	✓
S33	POST	`/v1/graphql`	—	✓	✓	✓	✓	✓
S34	GET	`/v1/catalog`	✓	—	—	—	—	—
S35	GET	`/v1/bowl-options`	✓	—	—	—	—	—

ID	HTTP	Ruta	N	C	A	K	D	O
S36	GET	`/v1/admin/inventory`	—	—	✓	—	—	✓
S37	POST	`/v1/admin/products`	—	—	✓	—	—	✓
S38	PUT	`/v1/admin/products/{item_id}`	—	—	✓	—	—	✓
S39	PUT	`/v1/admin/inventory/{item_id}/availability`	—	—	✓	—	—	✓
S40	POST	`/v1/admin/inventory/{item_id}/adjustments`	—	—	✓	—	—	✓
S41	GET	`/auth/me`	—	✓	✓	✓	✓	✓
S42	GET	`/health`	✓	—	—	—	—	—

5 Clases del API Gateway



Las líneas continuas representan asociaciones estructurales. La aplicación del Gateway tiene un router principal. Cada `ProxyRoute` pertenece a ese router en el registro del Gateway, y el router tiene una o más instancias de `ProxyRoute` en la configuración entregada. También contiene rutas propias, como salud e identidad, fuera de esa asociación. Para cada operación reenviada, `TARGETS` vincula su función con un destino, que puede atender varias operaciones. Las flechas discontinuas indican dependencias. Los bloques de identidad, destino y secretos agrupan funciones o configuración del código.

Se publica un Gateway HTTP para rutas anónimas, autenticadas y administrativas. El Gateway registra operaciones explícitas a partir de los routers. `ProxyRoute` valida el Bearer, introspecta cuando la ruta lo exige y verifica el rol antes de reenviar. No reenvía cabeceras de identidad del solicitante. Construye `X-PF-User` y `X-PF-Role` con los datos de introspección y añade el secreto del destino. Cada servicio vuelve a validar la credencial técnica y el rol; el dominio aplica propiedad, estado y DTO. Para operaciones internas, `PrivateRoute` también comprueba que el servicio llamante esté autorizado. GraphQL es un segundo gateway especializado y privado. `/auth/introspect` y `/internal/health` quedan fuera del enrutamiento externo, al igual que las rutas `/internal/` del módulo 6. Se registran método, ruta de plantilla, estado y correlación sin cuerpos ni cabeceras de autenticación.

6 Rutas y secretos de integración

Las rutas concretas, verbos y destinos se detallan por operación en los módulos 1 y 4. El Gateway conserva sólo `Content-Type` y `Content-Disposition` de la respuesta privada, y devuelve su propia cabecera `X-Request-ID`. Los cuerpos grandes reciben 413, y un cuerpo con otro tipo de medio, 415. Las rutas anónimas de cuenta comparten un límite de laboratorio de 40 solicitudes por minuto por proceso. Las solicitudes que lo exceden reciben 429. HTTP 401 indica sesión ausente o inválida, 403 rol o credencial técnica insuficiente y 404 identificador inexistente o pedido ajeno. HTTP 503 informa una dependencia indisponible. Los errores de negocio conservan el código de origen. El timeout es de 4 s por fase HTTP, salvo Gateway a Orders, que usa 20 s para dar margen a la coordinación interna. La salud consulta cada servicio en secuencia.

Matriz consolidada de rutas

Todas las operaciones entran por el Gateway HTTP. En las rutas reenviadas, el destino recibe el mismo método y ruta. Las rutas S41 y S42 se resuelven en el propio Gateway. **Acceso:** N = anónimo, C = cliente, A = administrador, K = cajero, D = despacho y O = dueño. Las rutas protegidas usan Bearer e introspección con Auth. El código `to_<destino>` identifica la credencial interna que envía el Gateway y el destino verifica como `from_gateway`. Para S42, se usa una credencial por destino. Además de los controles por ruta, aplican las validaciones comunes descritas arriba.

ID	Método y ruta	Acceso	Destino	Control específico	Credencial interna
S01	POST `/auth/login`	N	auth	Límite de cuenta y validación	`to_auth`
S02	POST `/auth/logout`	C A K D O	auth	Invalida la sesión actual	`to_auth`
S03	POST `/auth/verify`	N	auth	Límite de cuenta y validación	`to_auth`
S04	POST `/auth/resend`	N	auth	Límite de cuenta y validación	`to_auth`
S05	POST `/auth/recovery`	N	auth	Límite de cuenta y validación	`to_auth`
S06	POST `/auth/reset`	N	auth	Límite de cuenta y validación	`to_auth`
S07	POST `/api/v1/admin/users`	A	auth	Rol y validación de cuenta interna	`to_auth`
S08	GET `/api/v1/admin/users`	A	auth	Rol y paginación	`to_auth`
S09	PUT `/api/v1/admin/users/{user_id}`	A	auth	Rol y validación de cuenta interna	`to_auth`
S10	POST `/api/v1/clients`	N	clients	Límite de cuenta y validación	`to_clients`
S11	POST `/api/v1/admin/clients`	A	clients	Rol y validación de perfil	`to_clients`
S12	GET `/api/v1/clients/me`	C	clients	Identidad del cliente	`to_clients`
S13	GET `/api/v1/admin/clients`	A	clients	Rol y paginación	`to_clients`
S14	GET `/api/v1/admin/clients/{client_id}`	A	clients	Rol e ID de cliente	`to_clients`
S15	PUT `/api/v1/admin/clients/{client_id}`	A	clients	Rol y validación de perfil	`to_clients`
S16	PUT `/api/v1/admin/clients/{client_id}/active`	A	clients	Rol y cambio de activación	`to_clients`
S17	GET `/api/v1/admin/mail/{recipient_id}`	A	clients	Rol y destinatario de la bandeja	`to_clients`

ID	Método y ruta	Acceso	Destino	Control específico	Credencial interna
S18	GET `/api/v1/delivery/zones`	N	orders	Proyección de zonas habilitadas	`to_orders`
S19	POST `/api/v1/orders/quote`	N	orders	Valida cobertura y cotiza stock	`to_orders`
S20	POST `/api/v1/orders`	C	orders	Identidad, cobertura y reserva idempotente	`to_orders`
S21	GET `/api/v1/orders`	C A	orders	Filtrado por rol y titular	`to_orders`
S22	GET `/api/v1/orders/{order_id}`	C A K D O	orders	Titularidad y proyección por rol	`to_orders`
S23	POST `/api/v1/orders/{order_id}/cancel`	C A	orders	Titularidad, estado y reposición	`to_orders`
S24	GET `/api/v1/dispatch`	A D	orders	Rol, estado y paginación	`to_orders`
S25	PUT `/api/v1/dispatch/{order_id}/state`	A D	orders	Rol y transición permitida	`to_orders`
S26	GET `/api/v1/dispatch/{order_id}/print`	A D	orders	Rol y estado de despacho	`to_orders`
S27	POST `/api/v1/orders/{order_id}/payment`	C	payments	Titularidad y estado del pedido	`to_payments`
S28	GET `/api/v1/orders/{order_id}/payment`	C A K O	payments	Titularidad y proyección por rol	`to_payments`
S29	POST `/api/v1/orders/{order_id}/payment/confirm`	K	payments	Rol cajero y confirmación idempotente	`to_payments`
S30	GET `/api/v1/orders/{order_id}/receipt`	C A K	payments	Acceso al pedido y boleta emitida	`to_payments`
S31	GET `/api/v1/reports/sales`	A O	payments	Rol y rango de fechas	`to_payments`
S32	GET `/api/v1/reports/sales/print`	A O	payments	Rol y rango de fechas	`to_payments`
S33	POST `/api/v1/graphql`	C A K D O	graphql	Esquema, límites y permisos por campo	`to_graphql`
S34	GET `/api/v1/catalog`	N	inventory	Filtros y proyección pública	`to_inventory`
S35	GET `/api/v1/bowl-options`	N	inventory	Opciones públicas del configurador	`to_inventory`
S36	GET `/api/v1/admin/inventory`	A O	inventory	Rol y paginación	`to_inventory`
S37	POST `/api/v1/admin/products`	A O	inventory	Rol y validación de producto	`to_inventory`
S38	PUT `/api/v1/admin/products/{item_id}`	A O	inventory	Rol y validación de producto	`to_inventory`
S39	PUT `/api/v1/admin/inventory/{item_id}/availability`	A O	inventory	Rol y disponibilidad del ítem	`to_inventory`
S40	POST `/api/v1/admin/inventory/{item_id}/adjustments`	A O	inventory	Rol y ajuste de existencias	`to_inventory`
S41	GET `/auth/me`	C A K D O	Gateway	Introspección de sesión	`to_auth`
S42	GET `/health`	N	Gateway	Salud de los seis servicios	`to_<destino>`

Contratos privados y autoridad por tramo

Los IDs de ruta se muestran como `:id`. Cada par llamante–destino tiene su propia credencial. El llamante usa `to_<destino>` y el receptor valida `from_<llamante>`. Las rutas privadas no aparecen en OpenAPI del Gateway.

Llamante y destino	Método y ruta	Finalidad
Gateway a Auth	POST `/auth/introspect`	Identidad vigente para rutas protegidas.
Gateway a servicios	GET `/internal/health`	Diagnóstico por servicio.
GraphQL a Orders y Payments	GET S22 y S28	Agregación de lectura con identidad original.
Clients a Auth	POST `/internal/clients` y PUT `/internal/clients/:id/active`	Registrar identidad o cambiar su activación.
Clients a Orders	POST `/internal/client-order-counts`	Conteo por ID de cliente sin registros provisionales.
Orders a Clients	GET `/internal/clients/:id/checkout`	Verificar cliente y obtener datos de contacto mínimos.
Payments a Clients	GET `/internal/clients/:id/receipt-contact`	Obtener correo para el aviso de la boleta.
Orders a Inventory	POST `/internal/quote` y POST `/internal/reservations/:id`	Cotizar y aplicar una reserva idempotente.
Orders a Inventory	POST `/internal/reservations/:id/release`	Reponer una sola vez.
Orders a Payments	POST `/internal/payments/:id/void`	Cerrar el intento, la venta y la boleta, según corresponda.
Payments a Orders	GET `/internal/orders/:id/payment`	Consultar pedido con la identidad original.
Payments a Orders	PUT `/internal/orders/:id/confirmation` y PUT `/internal/orders/:id/paid`	Registrar confirmación en curso y finalizar como pagado.

Inventario simbólico de secretos

Ruta o identidad	Referencia simbólica	Consumo, alcance y manejo
Inicializador hacia Vault	`VAULT_DEV_ROOT_TOKEN_ID`	Valor efímero generado por el iniciador y entregado a Vault por variable de entorno. Los procesos de API reciben exclusivamente sus tokens acotados.
Cada proceso hacia Vault	PF_VAULT_TOKEN	Token de inicio temporal, política de lectura en una sola ruta `secret/data/pokefresh/<servicio>`. La app lee el valor al arrancar y lo mantiene en memoria.
Gateway hacia auth	`to_auth` / `from_gateway`	Login, flujos de cuenta y `POST /auth/introspect`. Envío mediante `X-PF-Secret` junto a identidad del componente.
Gateway hacia cada dominio y GraphQL	`to_<destino>` / `from_gateway`	Una clave aleatoria por destino. Rutas públicas seleccionadas y salud privada. Un token del navegador carece de valor como secreto técnico.

Ruta o identidad	Referencia simbólica	Consumo, alcance y manejo
GraphQL hacia orders/payments	`to_orders`, `to_payments` / `from_graphql`	Claves diferentes de las del Gateway HTTP. Autorizan sólo S22 y S28, con identidad original y correlación.
Servicios entre sí	`to_<destino>` / `from_<llamante>`	Una clave por cada par autorizado. La política de la ruta limita qué servicio puede invocarla.
Admin inicial	ADMIN_PASSWORD	Clave aportada en PowerShell. Argon2id al crear la cuenta ficticia inicial. El cambio de variable no reemplaza el hash persistido.
MongoDB, correo y pago	Loopback, adaptadores locales	MongoDB de laboratorio escucha en loopback y carece de autenticación propia. El buzón local y el pago local no usan claves de proveedor.

En cada arranque del iniciador nativo se reprovisionan las claves internas y se inician juntos los procesos para evitar claves desalineadas. Los tokens de Vault expiran a los 20 min y se usan sólo durante el arranque. La caché vive en memoria de proceso. Si termina un proceso administrado, el iniciador cierra el conjunto. La rotación se realiza reiniciando el iniciador nativo. La configuración supone un equipo local de confianza. Vault dev carece de persistencia y el volumen MongoDB se conserva. El tráfico de la versión verificada usa HTTP local, sin TLS. T18 comprueba el rechazo directo sin credencial y el 404 de la ruta de cotización privada por el Gateway.

7 Datos sensibles y secretos del cliente

Clase	Datos	Control y exposición
P, público	IDs, nombres, descripciones, precios, categoría, disponibilidad de catálogo y destinos ficticios.	DTO público excluye cantidad real de stock y secretos. La cotización anónima opera sobre SKUs y destinos de prueba.
R, restringido	RUN, nombre, nacimiento, sexo, teléfono, correo, domicilio, historial, motivos, venta, stock y documentos.	Verificación de rol y propiedad. Proyecciones limitadas por contexto. Dueño consulta reporte agregado; cajero recibe pedido sin domicilio; despacho recibe datos para preparar o entregar.
S, secreto	Claves, tokens Bearer, códigos de verificación/recuperación, credenciales internas.	Argon2id para contraseñas; huella SHA-256 para token opaco y códigos en cuentas. KV v2 para claves de integración. Sin eco de entrada en fallos de validación.

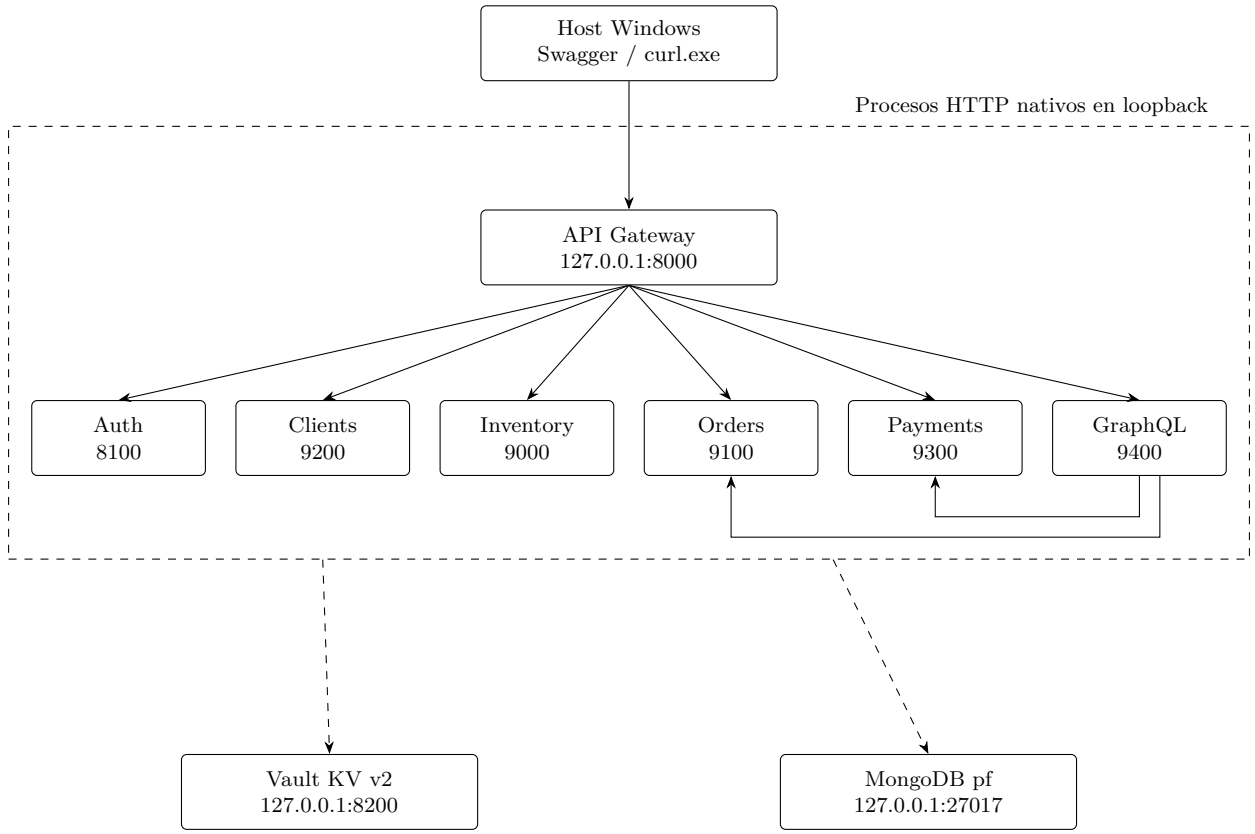
El token Bearer es una credencial de sesión del cliente, expira en 15 min, se revoca al salir y se invalida al cambiar contraseña, rol o estado de cuenta. Se guarda su hash en MongoDB. Se usa el campo Authorize de Swagger durante la sesión de laboratorio. El navegador no recibe claves de Vault, servicios o proveedores. El Gateway devuelve `Cache-Control: no-store`. La implementación comprueba dígito verificador del RUN, formato de correo, nombre completo, domicilio y división territorial no vacíos, fecha de nacimiento anterior a hoy, sexo según formulario y teléfono chileno. Las contraseñas se validan con longitud de 12–128. La base local conserva en claro los campos operativos personales bajo aislamiento de red, por lo que la demostración usa identidades ficticias. La escucha en loopback limita el acceso desde la red externa al equipo. Los servicios HTTP privados requieren además secretos técnicos. Cualquier proceso del equipo con acceso al puerto local de MongoDB puede consultar la base de laboratorio.

Correo local. El alta directa y administrativa crean un código de un uso por 30 minutos. Se guarda temporalmente en el buzón restringido para que un administrador de la demostración pueda copiarlo; la cuenta conserva sólo su hash. Canjearlo elimina el mensaje y habilita la sesión. Los códigos expirados quedan excluidos del buzón y se eliminan por TTL. Recuperación usa el mismo adaptador y revoca sesiones al cambiar la clave. Un administrador de laboratorio puede leer un código de reset vigente. Esta capacidad queda restringida al entorno local con cuentas ficticias. La boleta genera un aviso persistente con destinatario y referencia. El flujo demuestra el canje de un desafío y el encolado local. La existencia del buzón externo y la entrega real quedan pendientes por falta de credenciales de mensajería.

Pago local. Se preservan los métodos del prototipo `webpay` y `transferencia`. Los escenarios de prueba producen un intento persistente autorizado o rechazado. La referencia se consulta con rol cajero y la confirmación compara referencia, decisión registrada, monto y moneda del pedido. El importe se origina en el servidor. El adaptador no solicita datos bancarios ni ejecuta un cobro o devolución externa. La boleta es un documento digital de demostración sin emisión tributaria. Los comprobantes de seguridad y los pasos de captura están en el módulo 10.

8 Separación privada y agregaciones GraphQL

Mapa de despliegue editable



El Gateway, los seis procesos HTTP privados, MongoDB y Vault escuchan exclusivamente en loopback del equipo. El puerto 8000 es la entrada para solicitudes de usuario. Desde el mismo equipo se puede intentar un acceso directo a un puerto privado para comprobar que rechaza las llamadas HTTP sin credencial técnica. El iniciador ejecuta los binarios nativos, configura la réplica local y conserva los datos bajo `.local/mongo`. El reporte y la orden imprimible son respuestas PDF de la API.

```
type Query { pedido(id: ID!): Pedido }
type Pedido { id: ID!, estado: String!, total: Int!, moneda: String!, pago: Pago }
type Pago { estado: String!, monto: Int, metodo: String, referencia: String }
```

El resolver `pedido` consulta orders y hereda la verificación de propietario. El resolver `pago` consulta Payments para el mismo ID. Payments consulta a Orders por el contrato privado que aplica la propiedad. Ambos resolvers usan datos persistentes. Despacho recibe error de campo si solicita pago; `referencia` sólo se resuelve para el cajero. El esquema carece de campos de domicilio, correo o RUN. Hay una operación y una selección raíz por petición, máximo de 2.500 caracteres, 160 tokens de sintaxis, 24 selecciones, profundidad 4, hasta 5 variables de texto y 64 caracteres por valor. Se bloquean alias, fragmentos, directivas, mutaciones e introspección GraphQL. Una ejecución exitosa o con error de campo retorna 200 con `data` y, si aplica, `errors`; una consulta rechazada por forma o límites de estructura retorna 400. Superar el máximo de 2.500 caracteres devuelve 422. Las lecturas HTTP son independientes; un cambio de estado simultáneo puede observarse entre ambas y la vista puede consultarse de nuevo. El contrato JSON y la demostración desde Gateway figuran en S33 y en E15–E16.

9 Analítica y comunicación gRPC cuando correspondan

Decisión	Necesidad y carga prevista	Alternativa implementada y criterio de revisión
ADR-01. Reporte transaccional sin servicio analítico	El dueño consulta ventas de un local. Hipótesis de laboratorio: 100 pedidos/día y hasta 36.600 registros en 366 días. Reporte ocasional, sin cruces históricos externos.	Agregación MongoDB por índice de fecha y ventana máxima de 366 días. S31/S32 exponen ventas brutas, anuladas, netas y ticket promedio. E17 y E21. Revisar si consultas repetidas compiten con las ventas o se piden cruces entre locales.
ADR-02. HTTP interno para GraphQL, sin gRPC	Operaciones unitarias y mensajes pequeños. Hipótesis de pico: 5 solicitudes GraphQL/s, hasta 15 consultas de negocio internas/s más introspección si se solicita el pago. No se necesita streaming ni contrato binario.	HTTP JSON explícito con credenciales por destino, timeout y proyecciones. E15, E16, E20 y E21 comprueban la alternativa funcional. Revisar ante flujo continuo de eventos o requisitos de latencia que HTTP no cubra según medición.
ADR-03. Bandeja local sin STOMP/ActiveMQ	El sistema necesita registrar el aviso y consultar la cola de despacho. No hay un consumidor externo en el entorno local.	Bandeja persistente y orden de despacho por pago. Crear el aviso con la misma transacción evita perderlo tras confirmar la venta. La entrega real y los reintentos de mensajería requerirían un consumidor.

El reporte usa ventas cuya fecha de creación cae en el rango UTC. Las anulaciones posteriores afectan el neto del período de la venta original. El promedio se expresa en CLP enteros. Las cifras de demanda son supuestos. Las mediciones y los objetivos de laboratorio se detallan en E21. E21 ejecuta dos lotes de 1.000 solicitudes, con dos ventas y concurrencia 1. La proyección de 36.600 registros queda sin medir. El sistema no usa streaming.

Medición de laboratorio en Windows

Se enviaron 1.000 solicitudes de GraphQL S33 y 1.000 del reporte S31, en dos lotes secuenciales con concurrencia 1 y dos ventas en el período. Se observó la latencia desde PowerShell, incluido `curl.exe`. Se midieron nueve procesos de servidor en cada lote.

Métrica	GraphQL	Reporte
Solicitudes	1.000	1.000
Duración	206,12 s	207,51 s
Ritmo observado	4,85 solicitudes/s	4,82 solicitudes/s
p95 observado	48,93 ms	36,81 ms
Objetivo p95	$\leq$ 500 ms	$\leq$ 1.000 ms
Errores	0	0
CPU acumulada	25,30 s	15,09 s
CPU media sobre capacidad total del equipo	0,77 %	0,45 %
Muestras de memoria	382	385
Pico observado de la suma de conjuntos de trabajo	829,36 MiB	831,55 MiB

Ambos lotes cumplieron los objetivos de laboratorio (E21). La CPU se obtuvo por diferencia de tiempo de proceso. La memoria corresponde a la suma de los conjuntos de trabajo muestreados cada 500 ms y puede incluir páginas compartidas más de una vez. Estos resultados respaldan la alternativa HTTP y el reporte transaccional para el escenario medido. Queda por medir el comportamiento con más ventas y mayor concurrencia.

10 Verificación del backend

Backend evaluado. El código de la API y este informe se entregan en el mismo ZIP, junto con `run.py`, `requirements.txt`, `catalog.json`, `README.md` y `pruebas.py`. El README describe cómo iniciar el backend local y ejecutar las comprobaciones básicas. La ejecución en Windows documentada en este informe corresponde a la versión en que cada servicio controla la escritura de sus propios datos. Las cuentas, los destinos, el pago y el buzón de laboratorio son ficticios.

El backend usa FastAPI, Uvicorn y MongoDB mediante PyMongo. La autenticación usa Argon2id y Vault administra las credenciales técnicas. La agregación de pedidos y pagos se implementa con GraphQL Core.

Verificación. Se ejecutó el guion de PowerShell en Windows con comprobaciones de código HTTP, contenido y efectos de negocio. La revisión de los PDF se realizó visualmente. El comportamiento de los reintentos ante fallos simulados se comprobó por separado con pruebas aisladas. Las evidencias adjuntas documentan la corrida en Windows. El guion utilizado y las pruebas aisladas no forman parte de los archivos adjuntos. La indisponibilidad entre procesos reales quedó fuera de esa corrida.

Datos de prueba

El README describe el arranque local. Para reproducir la corrida se crean los roles `cajero`, `despacho` y `dueno` con S07 y dos clientes ficticios con RUN de formato válido y correos bajo `example.test`. Las pruebas usan los IDs devueltos por la API y la credencial del rol correspondiente. Las capturas se incluyen en el directorio `Evidencias`. En el PDF de despacho se ocultaron los datos de contacto de demostración.

Matriz de pruebas y evidencias

Requisito	Pantalla	Servicio	Prueba	Resultado	Evidencia
R08	P08	S01 / S07	T00 Alta y acceso de roles internos	Conforme	E00
R12	P12	S42	T01 Arranque y OpenAPI	Conforme	E01
R01/R02	P01/P02	S34–35	T02 Catálogo y configurador	Conforme	E02
R03/R08	P03/P08	S01 / S03 / S10 / S11 / S17	T03 Alta y correo local	Conforme	E03
R03	P03	S01–06 / S41	T04 Sesión y recuperación	Conforme	E04
R04	P04	S18–19	T05 Cobertura y cotización	Conforme	E05
R04	P04	S20 / S36	T06 Pedido y reserva idempotente	Conforme	E06
R05	P05	S21–22	T07 Pedido ajeno	Conforme	E07
R04/R10	P04/P10	S25 / S27–29	T08 Preparación anticipada y pago rechazado	Conforme	E08
R04/R06	P04/P06	S27–29	T09 Confirmación y repetición	Conforme	E09
R06	P06	S17 / S30	T10 Boleta y aviso	Conforme	E10
R10	P10	S24–26	T11 Preparar, despachar e imprimir	Conforme	E11
R05	P05	S20 / S23 / S27–30 / S36	T12 Anular y devolver stock	Conforme	E12
R07	P07	S36–40	T13 Mantenedor y disponibilidad	Conforme	E13
R08	P08	S07–09 / S13–16	T14 Mantenedores de clientes y usuarios	Conforme	E14
R11	P11	S33	T15 GraphQL agrega dos subsistemas	Conforme	E15
R11	P11	S33	T16 Límites y permisos por campo	Conforme	E16
R09	P09	S31–32	T17 Reporte y exportación	Conforme	E17
R12	P12	S34 / S36	T18 Servicios privados e identidad falsa	Conforme	E18
R02/R03/ R04	P02/P03/ P04	S10 / S19 / S39	T19 Entradas inválidas y disponibilidad	Conforme	E19
R12	P12	S20 / S22 / S28 / S36 / S42	T20 Recarga, detención y persistencia	Conforme*	E20
R09/R11	P09/P11	S31 / S33	T21 Demanda de laboratorio	Conforme	E21
R04	P04	S20 / S23 / S36	T22 Checkout concurrente	Conforme	E22

Lectura de la matriz. Conforme identifica los escenarios observados en Windows para la versión del módulo 8. E00–E22 agrupan las capturas incluidas en el informe. *T20 registra la recarga y el reinicio; la captura intermedia del cierre queda sin adjuntar.

Resultados observados y evidencias

T00. Cuentas internas. El administrador inició sesión con HTTP 200. Las altas de cajero, despacho y dueño devolvieron HTTP 201. Los tres roles iniciaron sesión con HTTP 200 y aparecieron activos.

E00. Cuentas internas

```
Clave inicial del admin de AAA: ************
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /api/v1/admin/users -> HTTP 201 (esperado 201)
POST /api/v1/admin/users -> HTTP 201 (esperado 201)
POST /api/v1/admin/users -> HTTP 201 (esperado 201)
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token

nombre          role     active
------          ----     ------
Cuenta dueno    dueno      True
Cuenta despacho despacho   True
Cuenta cajero   cajero     True
```

T01. Arranque y OpenAPI. La instalación pasó `pip check`. `GET /health` devolvió HTTP 200 y los seis servicios figuraron en `ok`. OpenAPI incluyó GraphQL. Todas las escuchas mostradas estuvieron en `127.0.0.1`, con el Gateway en el puerto 8000.

E01. Arranque y OpenAPI

```
HTTP/1.1 200 OK
date: Mon, 09 Nov 2026 23:13:57 GMT
server: uvicorn
content-length: 131
content-type: application/json
x-request-id: 02d6b775f6db4fa9b7e8545ba19570ba
cache-control: no-store
x-content-type-options: nosniff

{"status": "ok", "services": {"auth": "ok", "inventory": "ok", "orders": "ok", "clients": "ok", "payments": "ok", "graphql": "ok"}}PokÃ© Fresh API Gateway
GraphQL aparece en OpenAPI: True

LocalAddress LocalPort OwningProcess
------------ --------- -------------
127.0.0.1        8000         25412
127.0.0.1        8100          1828
127.0.0.1        8200         17620
127.0.0.1        8201         17620
127.0.0.1        9000         29924
127.0.0.1        9100         31560
127.0.0.1        9200          4484
127.0.0.1        9300         24732
127.0.0.1        9400         30388
127.0.0.1       27017         28424
```

T02. Catálogo y configurador. El catálogo mostró 12 productos y ofertas. Las tres ofertas disponibles registraron precios promocionales y normales de 5.990/7.490, 5.790/7.290 y 4.990/6.390 CLP. El bowl devolvió un precio base de 4.990 CLP, con 4 bases, 4 proteínas, 7 toppings y 4 salsas.

E02. Catálogo y configurador

```
id      precio precioNormal disponible
--      ------ ------------ ----------
esp-001   5990         7490       True
esp-002   5790         7290       True
esp-003   4990         6390       True



productos_y_ofertas : 12
ofertas             : 3
precio_base_bowl    : 4990
bases               : 4
proteinas           : 4
toppings            : 7
salsas              : 4
```

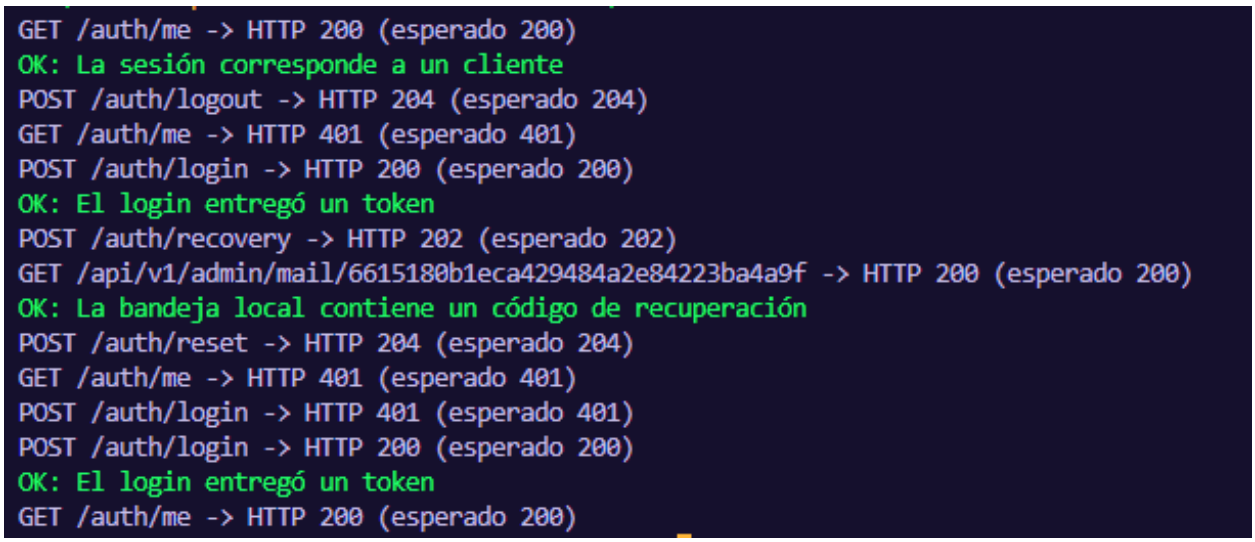
T03. Alta y correo local. Las altas directa y administrativa devolvieron HTTP 201. Antes del canje, ambas cuentas recibieron HTTP 403 al iniciar sesión. El reenvío devolvió HTTP 202, los dos canjes HTTP 200 y el reuso HTTP 400. Después del canje, ambos inicios de sesión devolvieron HTTP 200.

E03. Alta y correo local

```
POST /api/v1/clients -> HTTP 201 (esperado 201)
POST /api/v1/admin/clients -> HTTP 201 (esperado 201)
OK: Ambas altas quedan pendientes de verificación
POST /auth/login -> HTTP 403 (esperado 403)
POST /auth/login -> HTTP 403 (esperado 403)
POST /auth/resend -> HTTP 202 (esperado 202)
GET /api/v1/admin/mail/6615180b1eca429484a2e84223ba4a9f -> HTTP 200 (esperado 200)
GET /api/v1/admin/mail/318082ae8d44415a9043084feacf964f -> HTTP 200 (esperado 200)
OK: Ambas cuentas tienen código en la bandeja local
POST /auth/verify -> HTTP 200 (esperado 200)
POST /auth/verify -> HTTP 200 (esperado 200)
OK: Ambas cuentas quedaron verificadas en el adaptador local
POST /auth/verify -> HTTP 400 (esperado 400)
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
```

T04. Sesión y recuperación. Logout devolvió HTTP 204 y el mismo token recibió HTTP 401. Recuperación devolvió HTTP 202 y reset HTTP 204. El token y la clave anteriores recibieron HTTP 401. Con la clave nueva se obtuvo HTTP 200. La expiración por tiempo no se probó en este caso.

E04. Sesión y recuperación



T05. Cobertura y cotización. El destino ficticio a 1,112 km obtuvo HTTP 200, con 6.990 CLP por el producto, 6.190 CLP por el bowl, 13.180 CLP de total y despacho gratuito. El destino a 6,6717 km obtuvo HTTP 422 por exceder el radio de 3 km.

E05. Cobertura y cotización

```
GET /api/v1/delivery/zones -> HTTP 200 (esperado 200)

destino_id  distance_km in_coverage shipping
----------  ----------- ----------- --------
cobertura-1       1,112        True        0
fuera-1          6,6717       False


POST /api/v1/orders/quote -> HTTP 200 (esperado 200)

tipo     nombre             cantidad subtotal
----     ------             -------- --------
producto Clásico Hawaiano          1     6990
bowl     Bowl personalizado        1     6190



OK: Total CLP 13180 y despacho gratuito
POST /api/v1/orders/quote -> HTTP 422 (esperado 422)
total moneda despacho
----- ------ --------
13180 CLP           0




detail : Dirección fuera del radio de 3 km
```

T06. Pedido y reserva idempotente. El alta y su reintento secuencial devolvieron HTTP 201 con el mismo ID y estado `pendiente_pago`. El stock de `prod-001` pasó de 40 a 39 y permaneció en 39. Reusar la clave con un cuerpo distinto devolvió HTTP 409. Véase T22 para concurrencia.

E06. Pedido y reserva idempotente

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
POST /api/v1/orders -> HTTP 201 (esperado 201)
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
POST /api/v1/orders -> HTTP 201 (esperado 201)
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001

id                                state          total
--                                -----          -----
094b1181096645e7842bf08188484842 pendiente_pago 13180
094b1181096645e7842bf08188484842 pendiente_pago 13180


OK: El pedido nace pendiente de pago
OK: La repetición devolvió el mismo pedido
OK: Stock observado: 40 -> 39 -> 39
POST /api/v1/orders -> HTTP 409 (esperado 409)


detail : Clave de idempotencia usada con otro pedido
```

T07. Pedido ajeno. El titular obtuvo HTTP 200. El segundo cliente recibió HTTP 404 sin datos del pedido. Su historial devolvió HTTP 200 y omitió el pedido ajeno.

E07. Pedido ajeno

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/orders/094b1181096645e7842bf08188484842 -> HTTP 200 (esperado 200)
OK: El titular puede consultar su pedido
GET /api/v1/orders/094b1181096645e7842bf08188484842 -> HTTP 404 (esperado 404)

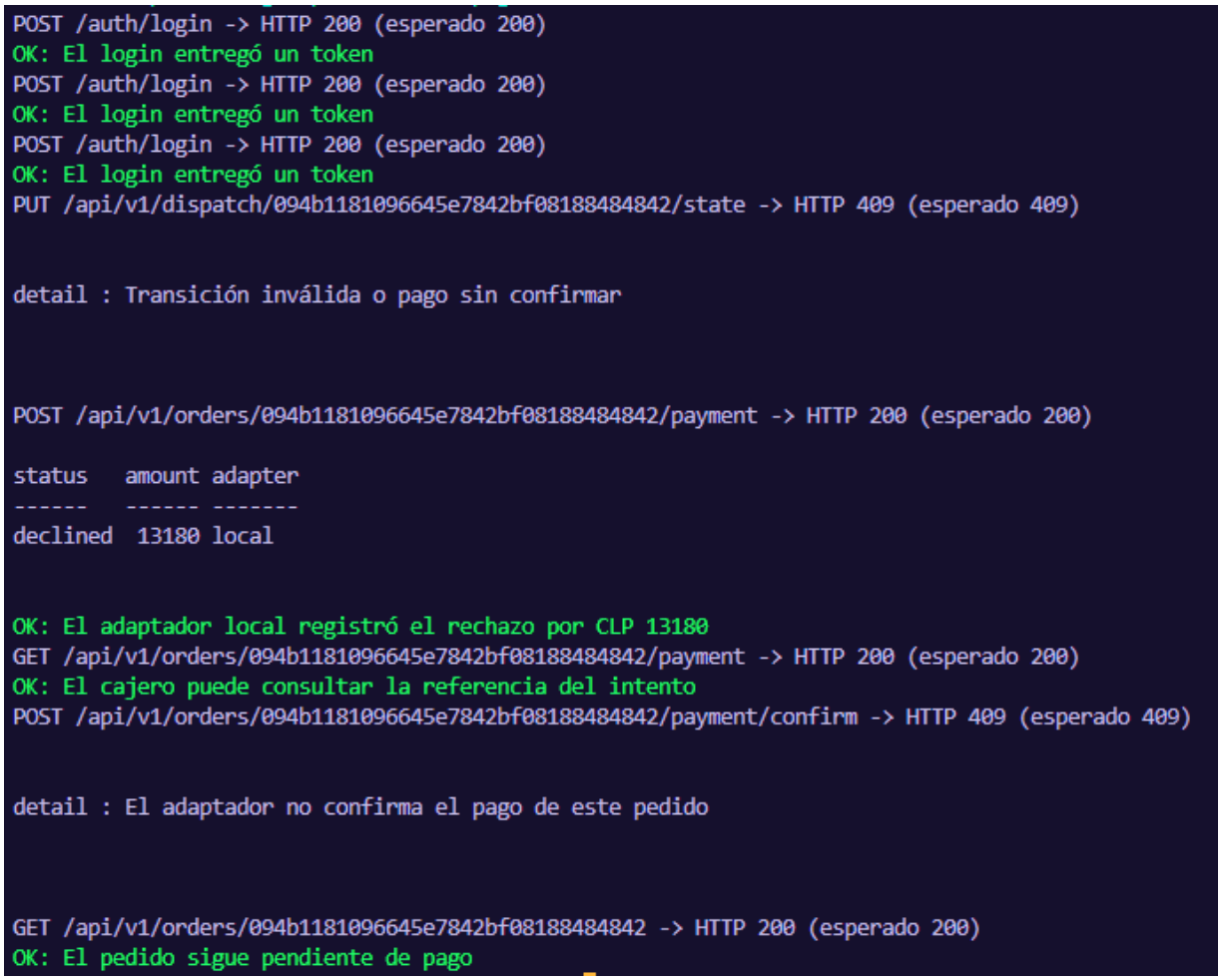

detail : Pedido no encontrado



OK: El rechazo no devuelve el ID del pedido
GET /api/v1/orders -> HTTP 200 (esperado 200)
OK: El historial del segundo cliente omite el pedido ajeno
```

T08. Preparación anticipada y pago rechazado. Preparar antes del pago devolvió HTTP 409. El intento local de 13.180 CLP se registró con HTTP 200 y estado `declined`. La consulta del cajero obtuvo HTTP 200 y la confirmación HTTP 409. El pedido permaneció en `pendiente_pago`.

E08. Preparación anticipada y pago rechazado



T09. Confirmación y repetición. El adaptador local autorizó 13.180 CLP. La confirmación como cliente, con referencia falsa y con monto extra devolvió HTTP 403, 409 y 422, respectivamente. El cajero obtuvo HTTP 200 en ambas confirmaciones válidas, con los mismos IDs. La repetición devolvió `duplicate=true` y el pedido quedó pagado.

E09. Confirmación y repetición

```
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment -> HTTP 200 (esperado 200)
OK: El adaptador local autorizó CLP 13180
GET /api/v1/orders/094b1181096645e7842bf08188484842/payment -> HTTP 200 (esperado 200)
OK: El cajero obtuvo la referencia vigente
1. Confirmación con rol cliente
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment/confirm -> HTTP 403 (esperado 403)
2. Confirmación con referencia falsa
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment/confirm -> HTTP 409 (esperado 409)
3. Confirmación con un monto adicional en el cuerpo
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment/confirm -> HTTP 422 (esperado 422)
4. Confirmación válida y repetición
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment/confirm -> HTTP 200 (esperado 200)
POST /api/v1/orders/094b1181096645e7842bf08188484842/payment/confirm -> HTTP 200 (esperado 200)

payment_status duplicate sale_id                        receipt_id
-------------- --------- -------                        ----------
confirmed          False 094b1181096645e7842bf08188484842 PF-094b1181096645e7842bf08188484842
confirmed           True 094b1181096645e7842bf08188484842 PF-094b1181096645e7842bf08188484842


OK: La segunda confirmación se identificó como duplicada
OK: Las dos respuestas conservan la misma venta y boleta
GET /api/v1/orders/094b1181096645e7842bf08188484842 -> HTTP 200 (esperado 200)
OK: El pedido quedó pagado
```

T10. Boleta y aviso. El titular obtuvo HTTP 200 para la boleta de demostración por 13.180 CLP, con el mismo número de la confirmación. La bandeja mostró un solo aviso local asociado. Otro cliente recibió HTTP 404.

E10. Boleta y aviso

```
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/orders/094b1181096645e7842bf08188484842/receipt -> HTTP 200 (esperado 200)


number        : PF-094b1181096645e7842bf08188484842
order_id      : 094b1181096645e7842bf08188484842
total         : 13180
document_type : boleta_demo



OK: La boleta coincide con la confirmación y el total de CLP 13180
OK: La boleta está identificada como demo
GET /api/v1/admin/mail/6615180b1eca429484a2e84223ba4a9f -> HTTP 200 (esperado 200)


kind       : receipt
adapter    : local
receipt_id : PF-094b1181096645e7842bf08188484842
order_id   : 094b1181096645e7842bf08188484842



OK: Hay un solo aviso local para la misma boleta
GET /api/v1/orders/094b1181096645e7842bf08188484842/receipt -> HTTP 404 (esperado 404)


detail : Pedido no encontrado
```

T11. Preparar, despachar e imprimir. El pedido pagado apareció en la cola con HTTP 200. La descarga del PDF devolvió HTTP 200 y mostró el mismo ID, un Clásico Hawaiano y un bowl personalizado. Las transiciones a `preparando`, `en_camino` y `entregado` devolvieron HTTP 200. La anulación durante la preparación devolvió HTTP 409.

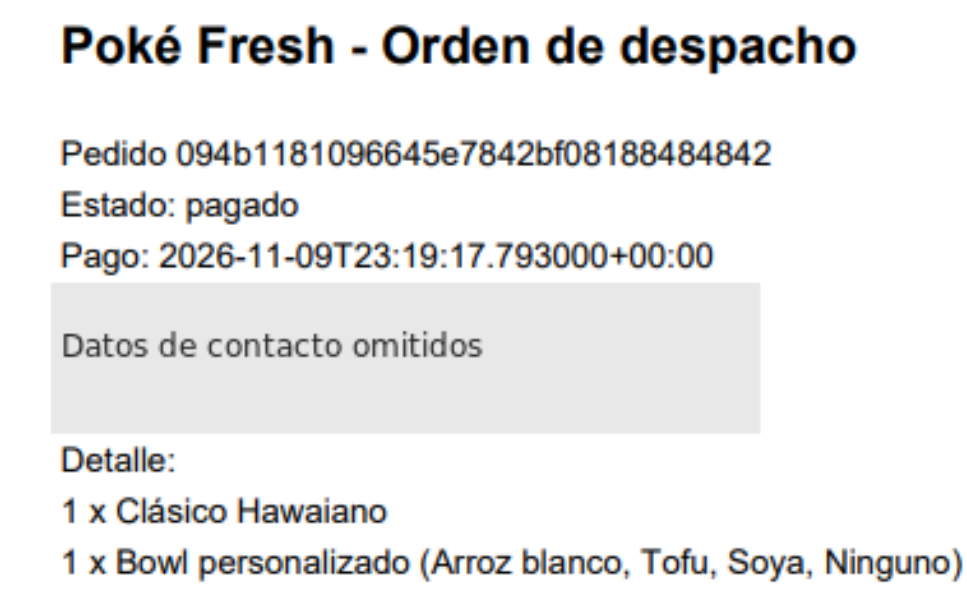
E11. Preparar, despachar e imprimir

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/dispatch -> HTTP 200 (esperado 200)

id                               state
--                               -----
094b1181096645e7842bf08188484842 pagado


OK: El pedido pagado aparece en la cola
GET /api/v1/dispatch/094b1181096645e7842bf08188484842/print -> HTTP 200 (esperado 200)
```

E11. Preparar, despachar e imprimir(continuación)



E11. Preparar, despachar e imprimir(continuación)

```
PUT /api/v1/dispatch/094b1181096645e7842bf08188484842/state -> HTTP 200 (esperado 200)
POST /api/v1/orders/094b1181096645e7842bf08188484842/cancel -> HTTP 409 (esperado 409)


detail : Sólo se anula antes de preparar y fuera de una confirmación en curso



PUT /api/v1/dispatch/094b1181096645e7842bf08188484842/state -> HTTP 200 (esperado 200)
PUT /api/v1/dispatch/094b1181096645e7842bf08188484842/state -> HTTP 200 (esperado 200)

id                               state
--                               -----
094b1181096645e7842bf08188484842 preparando
094b1181096645e7842bf08188484842 en_camino
094b1181096645e7842bf08188484842 entregado
```

T12. Anular y devolver stock. Ambas anulaciones del segundo pedido devolvieron HTTP 200 y se conservó el primer motivo. El stock pasó de 39 a 38, volvió a 39 al anular y permaneció en 39 al repetir. El pago quedó `refunded` en el adaptador local y la boleta quedó marcada como anulada.

E12. Anular y devolver stock

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
POST /api/v1/orders -> HTTP 201 (esperado 201)
OK: Se creó un segundo pedido
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
POST /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/payment -> HTTP 200 (esperado 200)
GET /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/payment -> HTTP 200 (esperado 200)
POST /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/payment/confirm -> HTTP 200 (esperado 200)
OK: El segundo pago quedó confirmado
POST /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/cancel -> HTTP 200 (esperado 200)
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
POST /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/cancel -> HTTP 200 (esperado 200)
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
GET /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/payment -> HTTP 200 (esperado 200)
GET /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/receipt -> HTTP 200 (esperado 200)
OK: Stock observado: 39 -> 38 -> 39 -> 39
OK: El pedido quedó anulado y conservó el primer motivo
OK: El pago quedó reembolsado en el adaptador local y la boleta marcada como anulada
```

T13. Mantenedor y disponibilidad. El alta de `prod-demo` devolvió HTTP 201 con stock de 5. La edición dejó el precio en 6.700 CLP y el ajuste subió el stock a 7. Tras deshabilitarlo, el catálogo normal lo omitió y el ampliado lo mostró no disponible. El cliente recibió HTTP 403 en inventario administrativo.

E13. Mantenedor y disponibilidad

```
POST /api/v1/admin/products -> HTTP 201 (esperado 201)
OK: El producto se creó con stock 5
PUT /api/v1/admin/products/prod-demo -> HTTP 200 (esperado 200)
OK: El precio cambió a 6700 y el stock se mantuvo en 5
POST /api/v1/admin/inventory/prod-demo/adjustments -> HTTP 200 (esperado 200)
OK: El ajuste dejó el stock en 7
PUT /api/v1/admin/inventory/prod-demo/availability -> HTTP 200 (esperado 200)
GET /api/v1/catalog?categoria=veggie -> HTTP 200 (esperado 200)
GET /api/v1/catalog?categoria=veggie&available_only=false -> HTTP 200 (esperado 200)
OK: El catálogo normal omite el producto deshabilitado
OK: El catálogo ampliado muestra el producto como no disponible
GET /api/v1/admin/inventory -> HTTP 403 (esperado 403)
```

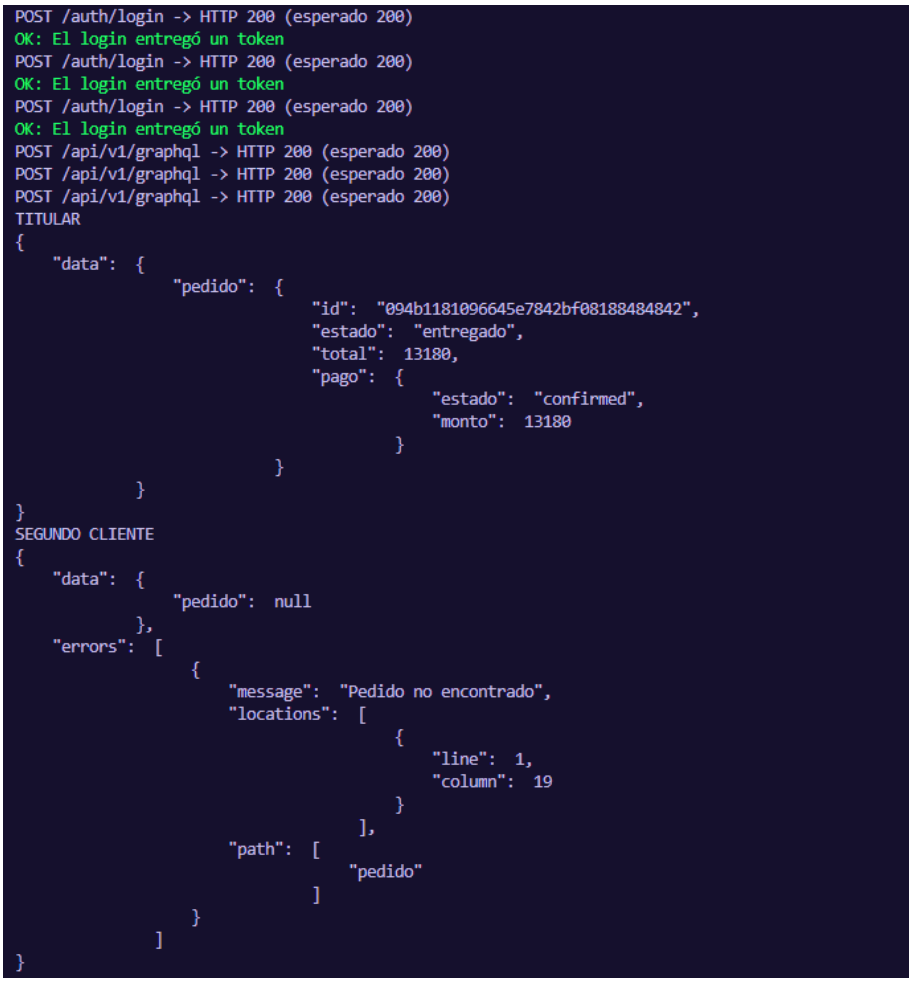
T14. Mantenedores de clientes y usuarios. El listado de clientes devolvió HTTP 200, con dos pedidos para el titular y cero para el segundo cliente. La edición actualizó el teléfono conservando RUN y correo. Las sesiones previas recibieron HTTP 401 tras editar la cuenta de despacho y desactivar al cliente. Reactivación, nuevo login y consulta de identidad devolvieron HTTP 200.

E14. Mantenedores de clientes y usuarios

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/users -> HTTP 200 (esperado 200)
OK: El administrador puede listar la cuenta de despacho
GET /auth/me -> HTTP 200 (esperado 200)
PUT /api/v1/admin/users/43a7a0919f4242aaa440679783c29c5d -> HTTP 200 (esperado 200)
GET /auth/me -> HTTP 401 (esperado 401)
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/clients -> HTTP 200 (esperado 200)
OK: Clients muestra 2 pedidos para el titular y 0 para el segundo cliente
GET /api/v1/admin/clients/318082ae8d44415a9043084feacf964f -> HTTP 200 (esperado 200)
GET /api/v1/clients/me -> HTTP 200 (esperado 200)
PUT /api/v1/admin/clients/318082ae8d44415a9043084feacf964f -> HTTP 200 (esperado 200)
GET /api/v1/clients/me -> HTTP 200 (esperado 200)
OK: El perfil refleja el teléfono nuevo y conserva RUN y correo
PUT /api/v1/admin/clients/318082ae8d44415a9043084feacf964f/active -> HTTP 200 (esperado 200)
GET /auth/me -> HTTP 401 (esperado 401)
PUT /api/v1/admin/clients/318082ae8d44415a9043084feacf964f/active -> HTTP 200 (esperado 200)
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /auth/me -> HTTP 200 (esperado 200)
```

T15. GraphQL agrega dos subsistemas. Las tres consultas devolvieron HTTP 200. El titular obtuvo pedido entregado y pago confirmado por 13.180 CLP. El segundo cliente recibió `pedido: null` con un error. Despacho obtuvo el pedido con `pago: null` y un error de autorización.

E15. GraphQL agrega dos subsistemas



E15. GraphQL agrega dos subsistemas(continuación)



T16. Límites y permisos por campo. Introspección, alias y múltiples operaciones devolvieron HTTP 400. Superar la longitud devolvió HTTP 422. La referencia de pago fue nula con error de campo para el cliente, mientras el cajero la obtuvo sin errores. Ambos casos devolvieron HTTP 200.

E16. Límites y permisos por campo

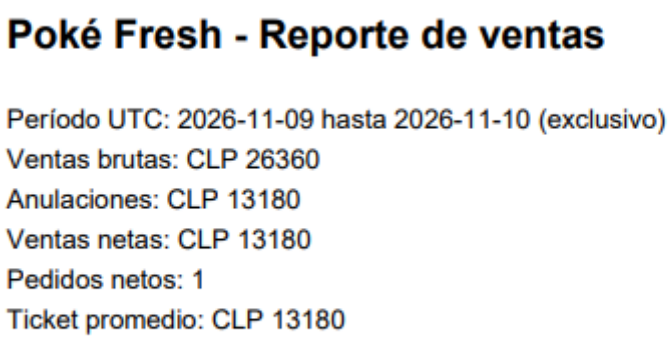


T17. Reporte y exportación. Para el 9 al 10 de noviembre de 2026 en UTC, con fin exclusivo, JSON y PDF devolvieron HTTP 200. Se observaron 26.360 CLP brutos, 13.180 CLP anulados y 13.180 CLP netos. El JSON registró un pedido neto y uno anulado. El cliente obtuvo HTTP 403 y el rango vacío HTTP 422.

E17. Reporte y exportación

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
Periodo UTC: 2026-11-09 hasta 2026-11-10 (exclusivo)
GET /api/v1/reports/sales?desde=2026-11-09&hasta=2026-11-10 -> HTTP 200 (esperado 200)
{
    "from":  "2026-11-09",
    "to_exclusive":  "2026-11-10",
    "currency":  "CLP",
    "gross":  26360,
    "voided":  13180,
    "net":  13180,
    "orders":  1,
    "voided_orders":  1,
    "average_ticket":  13180
}
OK: El reporte coincide con las dos ventas y una anulacion
GET /api/v1/reports/sales/print?desde=2026-11-09&hasta=2026-11-10 -> HTTP 200 (esperado 200)
GET /api/v1/reports/sales?desde=2026-11-09&hasta=2026-11-10 -> HTTP 403 (esperado 403)
GET /api/v1/reports/sales?desde=2026-11-09&hasta=2026-11-09 -> HTTP 422 (esperado 422)
```

E17. Reporte y exportación(continuación)



T18. Servicios privados e identidad falsa. Los dos accesos directos al catálogo devolvieron HTTP 403. Por el Gateway, una solicitud sin sesión devolvió HTTP 401; un cliente con rol falso, HTTP 403; y el administrador, HTTP 200. Introspección y cotización privada devolvieron HTTP 404 por el Gateway. Invocar `/internal/quote` en Inventory sin credencial técnica devolvió HTTP 403 incluso con Bearer de administrador.

E18. Servicios privados e identidad falsa

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
1. Catálogo directo sin credencial interna
GET http://127.0.0.1:9000/api/v1/catalog -> HTTP 403 (esperado 403)
2. Catálogo directo con cabeceras falsas
GET http://127.0.0.1:9000/api/v1/catalog -> HTTP 403 (esperado 403)
3. Gateway con rol falso y sin token
GET /api/v1/admin/inventory -> HTTP 401 (esperado 401)
4. Gateway con token de cliente y rol falso
GET /api/v1/admin/inventory -> HTTP 403 (esperado 403)
5. Control positivo con administrador real
GET /api/v1/admin/inventory -> HTTP 200 (esperado 200)
6. Introspección interna por el Gateway
POST /auth/introspect -> HTTP 404 (esperado 404)
7. Ruta privada nueva por el Gateway, incluso como admin
POST /internal/quote -> HTTP 404 (esperado 404)
8. Ruta privada nueva directa, con token de admin y sin credencial técnica
POST http://127.0.0.1:9000/internal/quote -> HTTP 403 (esperado 403)
```

T19. Entradas inválidas y disponibilidad. RUN inválido, cantidad cero, producto inexistente y toppings incompatibles devolvieron HTTP 422. El producto deshabilitado y la suma de cantidades que superó el stock disponible devolvieron HTTP 409.

E19. Entradas inválidas y disponibilidad



```
1. RUN con dígito verificador inválido
POST /api/v1/clients -> HTTP 422 (esperado 422)
{
    "detail":  [
                    {
                        "loc":  [
                                    "body",
                                    "run"
                                ],
                        "type":  "value_error"
                    }
                ]
}
2. Cantidad cero
POST /api/v1/orders/quote -> HTTP 422 (esperado 422)
{
    "detail":  [
                    {
                        "loc":  [
                                    "body",
                                    "items",
                                    0,
                                    "producto",
                                    "cantidad"
                                ],
                        "type":  "greater_than_equal"
                    }
                ]
}
3. Producto inexistente
POST /api/v1/orders/quote -> HTTP 422 (esperado 422)
{
    "detail":  "ID de producto inválido"
}
```

E19. Entradas inválidas y disponibilidad(continuación)



T20. Detención y persistencia. La recarga conservó 39 unidades. Tras reiniciar, la salud volvió a HTTP 200 y se recuperaron el pedido entregado por 13.180 CLP y el segundo pedido anulado con su reembolso local. Repetir el checkout con la clave original devolvió HTTP 201 y el mismo ID, manteniendo 39 unidades. E20 contiene el antes y el después del reinicio. No se adjunta una captura de la detención.

E20. Detención y persistencia

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
Carga inicial aplicada
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
OK: La recarga conservó el stock: 39
GET /api/v1/orders/094b1181096645e7842bf08188484842 -> HTTP 200 (esperado 200)

id                               state     total
--                               -----     -----
094b1181096645e7842bf08188484842 entregado 13180
```

E20. Detención y persistencia(continuación)

```
GET /health -> HTTP 200 (esperado 200)
OK: El Gateway recuperó el estado ok
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/orders/094b1181096645e7842bf08188484842 -> HTTP 200 (esperado 200)
OK: Se conservó el mismo pedido entregado por CLP 13180
GET /api/v1/orders/5650f9a23e794ea18aa9986df987dd94 -> HTTP 200 (esperado 200)
GET /api/v1/orders/5650f9a23e794ea18aa9986df987dd94/payment -> HTTP 200 (esperado 200)
OK: Se conservaron la anulación y el reembolso local
POST /api/v1/orders -> HTTP 201 (esperado 201)
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
OK: La clave anterior sigue devolviendo el mismo pedido
OK: El stock se conservó tras reiniciar y repetir el checkout: 39
```

T21. Demanda de laboratorio. Se ejecutaron 1.000 solicitudes por operación en Windows con dos ventas, concurrencia 1 y cero errores. Se registraron latencia, ritmo, CPU y memoria. Ambos p95 cumplieron los objetivos de laboratorio. La tabla del módulo 9 detalla la corrida.

E21. Demanda de laboratorio

```
prueba                    : GraphQL
solicitudes               : 1000
concurrencia              : 1
duracion_s                : 206,12
solicitudes_s             : 4,85
p95_ms                    : 48,93
errores                   : 0
procesos                  : 9
muestras_memoria          : 382
cpu_total_s               : 25,3
cpu_promedio_equipo_pct   : 0,77
memoria_pico_observado_MiB : 829,36

prueba                    : Reporte
solicitudes               : 1000
concurrencia              : 1
duracion_s                : 207,51
solicitudes_s             : 4,82
p95_ms                    : 36,81
errores                   : 0
procesos                  : 9
muestras_memoria          : 385
cpu_total_s               : 15,09
cpu_promedio_equipo_pct   : 0,45
memoria_pico_observado_MiB : 831,55
```

T22. Checkout concurrente. Se enviaron dos solicitudes con el mismo cuerpo y la misma clave sin esperar la primera respuesta. Ambas devolvieron HTTP 201 con el mismo ID. El stock pasó de 39 a 38 y volvió a 39 al anular el pedido de prueba con HTTP 200.

E22. Checkout concurrente

```
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
POST /auth/login -> HTTP 200 (esperado 200)
OK: El login entregó un token
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001

HTTP Pedido                           Estado         Detalle
---- ------                           ------         -------
 201 e70959aa234440f6b23bbc2837930e90 pendiente_pago
 201 e70959aa234440f6b23bbc2837930e90 pendiente_pago


GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
OK: Ambas solicitudes devolvieron HTTP 201 y el mismo pedido
OK: Se reservó una sola unidad: 39 -> 38
POST /api/v1/orders/e70959aa234440f6b23bbc2837930e90/cancel -> HTTP 200 (esperado 200)
OK: El pedido de prueba quedó anulado
GET /api/v1/admin/inventory?limit=200 -> HTTP 200 (esperado 200)
OK: Inventario contiene prod-001
OK: Stock observado: 39 -> 38 -> 39
```

Referencias

Caso 19 y prototipo responsivo de Poké Fresh. Material del curso sobre API Gateway, persistencia, GraphQL y mensajería. Lectura de secretos KV v2 y seguridad de GraphQL.